Specifying Uncertainty in Use Case Models

Man Zhang , Tao Yue , Shaukat Ali , Bran Selic , Oscar Okariz , Roland Norgre , Karmele Intxausti

PII: S0164-1212(18)30131-6
DOI: 10.1016/j.jss.2018.06.075
Reference: JSS 10184

To appear in: *The Journal of Systems & Software*

Received date: 21 June 2017
Revised date: 12 May 2018
Accepted date: 25 June 2018

Please cite this article as: Man Zhang , Tao Yue , Shaukat Ali , Bran Selic , Oscar Okariz , Roland Norgre , Karmele Intxausti , Specifying Uncertainty in Use Case Models, *The Journal of Systems & Software* (2018), doi: 10.1016/j.jss.2018.06.075

Highlights

- U-RUCM, a use case modeling framework particularly designed for modeling uncertainties.
- U-RUCM tool can be used for specifying use case models with uncertainties.
- U-RUCM was developed in the context of an EU project
- U-RUCM has been evaluated with two industrial case studies.

Specifying Uncertainty in Use Case Models

Man Zhang, Tao Yue, Shaukat Ali, Bran Selic
Oscar Okariz, Roland Norgre, Karmele Intxausti

Abstract

Context: Latent uncertainty in the context of software-intensive systems (e.g., Cyber-Physical Systems (CPSs)) demands explicit attention right from the start of development. Use case modeling—a commonly used method for specifying requirements in practice, should also be extended for explicitly specifying uncertainty.

Objective: Since uncertainty is a common phenomenon in requirements engineering, it is best to address it explicitly by identifying, qualifying, and, where possible, quantifying uncertainty at the beginning stage. The ultimate aim, though not within the scope of this paper, was to use these use cases as the starting point to create test-ready models to support automated testing of CPSs under uncertainty.

Method: We extend the Restricted Use Case Modeling (RUCM) methodology and its supporting tool to specify uncertainty as part of system requirements. Such uncertainties include those caused by insufficient domain expertise of stakeholders, disagreements among them, and known uncertainties about assumptions about the environment of the system. The extended RUCM, called U-RUCM, inherits the features of RUCM, such as automated analyses and generation of models, to mention but a few. Consequently, U-RUCM provides all the key benefits offered by RUCM (i.e., reducing ambiguities in requirements), but also, it allows specification of uncertainties with the possibilities of reasoning and refining existing ones and even uncovering unknown ones.

Results: We evaluated U-RUCM with two industrial CPS case studies. After refining RUCM models (specifying initial requirements), by applying the U-RUCM methodology, we successfully identified and specified additional 306% and 512% (previously unknown) uncertainty requirements, as compared to the initial requirements specified in RUCM. This showed that, with U-RUCM, we were able to get a significantly better and more precise characterization of uncertainties in requirement engineering.

Conclusion: Evaluation results show that U-RUCM is an effective methodology (with tool support) for dealing with uncertainty in requirements engineering. We present our experience, lessons learned, and future challenges, based on the two industrial case studies.

Keywords— Use Case Modeling; Belief; Uncertainty.

1 INTRODUCTION

The problem of uncertainty in software-intensive systems such as Cyber-Physical Systems (CPSs)), is familiar to the requirements engineering community. However, it has not been adequately addressed and, therefore, it lacks both methodological and tool support in both the literature and in practice. In their well-known use case modeling book, Bittner and Spence [1] pointed out that it is essential to take the time to fill out missing areas and drill down into uncertainty. Uncertainty can be due to diverse causes, such as insufficient domain expertise or lack of information. Given the significant increases in the complexity of modern CPS and the diversity of the environments in which they are deployed, it is

- *Man Zhang is a Ph.D. Student with Simula Research Laboratory and the University of Oslo, Oslo, Norway. E-mail: manzhang@ simula.no.*
- *Tao Yue is a Chief Research Scientist with Simula Research Laboratory and an Adjunct Associate Professor with the University of Oslo, Oslo, Norway. E-mail: tao@simula.no.*
- *Shaukat Ali is a Senior Research Scientist with Simula Research Laboratory, Oslo, Norway. E-mail: shaukat@simula.no.*
- *Bran Selic is a Visiting Scientist with Simula Research Laboratory, Oslo, Norwaya. E-mail: bselic @simula.no*
- *Oscar Okariz is Software Product Manager with ULMA Handling System, Oñati, Spain. E-mail: ookariz@manutencion.ulma.es*
- *Roland Norgre is a Process Manager R&I with Future Position X, Gävle, Sweden. E-mail: roland.norgren@fpx.se*
- *Karmele Intxausti is a Senior Researcher with Ikerlan, Arrasate-Mondragon, Spain. E-mail: KIntxausti@ikerlan.es*

becoming critical to address uncertainty up front; that is, right from the start of development. This includes not only uncertainties about the requirements, but also uncertainties about its assumed operating environment.

Uncertainty in requirements has been studied in the context of dynamically adaptive systems in the presence of environmental uncertainty [2, 3]. Several goal-driven solutions [4, 5] have been proposed to handle uncertainty in similar contexts. Partial model-based solutions (e.g., [6, 7]) have been developed to support early requirements and architecture decision making. However, after conducting a literature review, we did not find any use case modeling methodology that explicitly handles uncertainty. Having such a methodology is important since use case modeling is a commonly used technique for specifying requirements in practice [6]. In our view, because uncertainty is a common phenomenon in requirements engineering, it is best to *address it explicitly* by identifying, qualifying, and, where possible, quantifying uncertainty.

The need for such a methodology arose in the context of an EU Horizon 2020 project, which focused on testing CPSs under uncertainty. The crucial first step in this project was to collect use cases with known uncertainties for two industrial CPSs and their environments. This was done with three industrial partners (Future Position X[1], ULMA[2] and Ikerlan[3], which are among the authors of this paper). The ultimate aim was to use these use cases as the starting point to create test-ready models to support automated testing of CPSs under uncertainty. To this end, we first introduced the RUCM methodology to our industrial partners and then extended it to enable specification of uncertainties. This led to the design of the U-RUCM methodology, which, to the best of our knowledge, is the first use case modeling methodology that explicitly addresses uncertainty.

As noted, U-RUCM is based on a practical use case modeling solution, called Restricted Use Case Modeling (RUCM) [8, 9]. RUCM was initially proposed by Yue et al. [9], for reducing inherent ambiguity in textual *Use Case Specifications* (UCSs) and to enable automated generation of UML models. Later on, RUCM was extended to address various industrial challenges, including requirements based testing [10] and use case based requirements inspection [11]. RUCM and its extensions have been used to address industrial challenges from various domains (e.g., telecommunication [10, 12, 13], automotive [14]).

To structure and specify uncertainties in use case models, two templates were proposed for specifying *Belief Use Case Specifications* (BUCS) and uncertainties. A BUCS annotates the UCS with uncertainty information, including the source of uncertainty, the degree (measurement) of uncertainty, the risk of uncertainty, as perceived by stakeholders and based on available evidence. Such models can

[1] fpx.se/geo-sports/
[2] www.ulmahandling.com/en/
[3] www.ikerlan.es/

be automatically generated as instances of a formal U-RUCM metamodel.

Evaluation of U-RUCM based on the two industrial case studies revealed that, with U-RUCM, we were able to significantly improve on the characterization and understanding of uncertainties in the requirements (up to 306% and 512% for the two case studies) compared to base RUCM. In this paper, we summarize practical lessons learned in the course of this evaluation and also discuss future challenges.

The rest of the paper is organized as follows: Section 2 presents the background. The U-RUCM templates and restrictions rules including keywords are explained in Section 3, followed by its formalization (Section 4). The tool support and recommended methodology are given in Section 5. Section 6 reports user experience, evaluation, lessons learned, and future challenges. We discuss the related work in Section 7 and conclude the paper in Section 8.

2 BACKGROUND AND RUNNING EXAMPLE

In this section, we first briefly introduce the *U-Model* on which U-RUCM is based on, followed by the running example and a brief description of RUCM illustrated with the running example.

2.1 U-Model

To help us understand the nature of uncertainty in the general context of software engineering, in our previous work [15] we developed a conceptual model called *U-Model* to define uncertainty and its associated concepts. The *U-Model* was developed based on an extensive review of existing literature on uncertainty from several disciplines including philosophy, healthcare and physics, and two industrial case studies from the two industrial partners of the EU Horizon 2020 project. To keep the paper self-contained, we have provided *U-Model* and definitions of its concepts in Appendix A and in the rest of the section, we briefly summarize the fundamental concepts and their relationships.

The *U-Model* takes a subjective approach to representing uncertainty. This means that uncertainty is modeled as a state (i.e., worldview) of some agents (called *BeliefAgents*), who, for whatever reason, do not have complete and fully accurate knowledge about some subjects of interest. In the *U-Model*, a *Belief* is an abstract concept, but it can be expressed in the concrete form via one or more explicit *BeliefStatement*s (a concrete and explicit specification of some *Belief* held by a *BeliefAgent* about possible phenomena or notions belonging to a given subject area). Uncertainty (i.e., lack of confidence) represents a state of affairs whereby a *BeliefAgent* does not have full confidence in a belief that it holds. This may be due to any number of factors: lack of information, inherent variability in the subject matter, ignorance, or even due physical phenomena such as the Heisenberg uncertainty principle. While *Uncertainty* itself is an abstract concept, it can be quantified by a corresponding *Measurement*, which expresses in some concrete form the subjective degree of uncertainty that the agent ascribes to a

BeliefStatement. As the latter is a subjective notion, a *Measurement* should not be confused with the degree of validity of a *BeliefStatement*. Instead, it merely indicates the level of confidence that the agent has in a statement.

Some of the *U-Model* concepts were further extended for supporting Model-Based Testing (MBT) of CPSs under uncertainty. More specifically, we developed the Uncertainty Modeling Framework (*UncerTum*) [16, 17] for supporting MBT of CPSs, which contains a UML profile, called the UML Uncertainty Profile (UUP), for specifying and measuring uncertainties as part of UML models. UUP was derived based on the *U-Model*. *UncerTum* has been successfully used for discovering unknown uncertainties [16, 18] and generating test cases [19]. In addition, we recently developed the *UncerTolve* framework [18, 20] to evolve test ready models and uncertainty measurements in UML models specified with *UncerTum* with real operational data.

2.2 Running Example

We illustrate U-RUCM using a modified version of the SafeHome case study provided in [21]. The SafeHome system implements various security and safety features in smart homes, including intrusion detection, fire detection, and flooding. One of the key functionalities of the system is that a homeowner activates the monitoring function of the system, which continuously checks for intrusions until it is explicitly disabled. During monitoring, any occurrence of an intrusion should be detected, immediately followed by sending an intrusion notification to the homeowner and the activation of an alarm. The corresponding use case for this example, named *Monitor Windows and Doors*, is shown in Figure 1. In Section 2.3, we illustrate the key elements of RUCM with the running example.

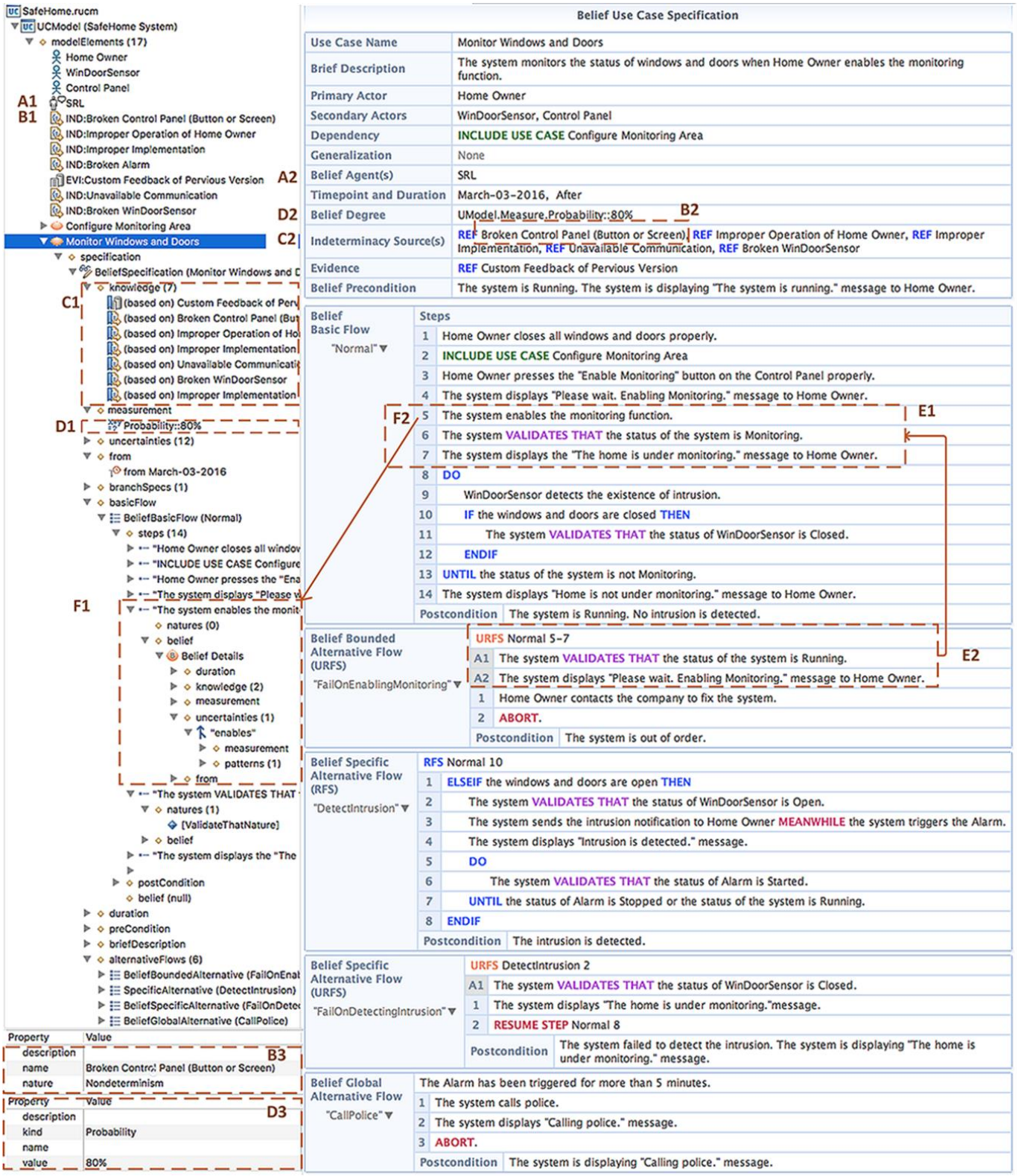
2.3 Restricted Use Case Modeling (RUCM)

RUCM encompasses a use case template and 26 restriction rules for specifying textual UCSs [9]. RUCM aims to be easy to use, to reduce ambiguity and improve understanding, and to facilitate automated analysis. Results of two controlled experiments support these expectations [8, 9]. A RUCM UCS has one basic flow and, optionally, one or more alternative flows. An alternative flow always depends on a condition occurring in a specific step of another flow (referred to as the reference flow). We classify alternative flows into three types: A *specific* alternative flow refers to a specific step in the reference flow; a *bounded* alternative flow refers to more than one step (consecutive or not) in the reference flow; a *global* alternative refers to any step in the reference flow. For specific and bounded alternative flows, a RFS (Reference Flow Step) section specifies one or more (reference flow) step numbers. For example, as shown in Figure 1, the use case has one basic flow, called *Normal*. The specific alternative flow of *DetectIntrusion* branches out from step 10 of the basic flow *Normal*, as indicated by keyword RFS and "Normal 10". The global alternative flow *CallPolice* is triggered

whenever the branching condition "The Alarm has been triggered for more than 5 minutes". The bounded alternative flow of *FailOnEnablingMonitoring* refers to steps 5-7 of the basic flow *Normal*, as indicated by "URFS Normal 5-7". URFS is a new keyword introduced to U-RUCM and will be discussed in Section 3.

RUCM defines a set of keywords to specify sentences that involve conditional logic (IF-THEN-ELSE-ELSEIF-ENDIF), concurrency (MEANWHILE), condition checking (VALIDATES THAT), and iteration (DO-UNTIL). For example, as shown in Figure 1, step 6 of the basic flow (*Normal*) contains the keyword VALIDATES THAT, implying that the sentence of the step is a condition checking sentence.

UCMeta is a metamodel that formalizes textual RUCM to facilitate automated analyses and generation of UML analysis models [8, 22]. UCMeta is specified using the OMG's standard Meta-Object Facility (MOF) [23], while the formalization of RUCM models to UCMeta instances is done automatically, as described in [8]. For example, as shown on the left panel of Figure 1, the basic flow *Normal* is formalized as an instance of metaclass *BasicFlow* (i.e., *basicFlow*).

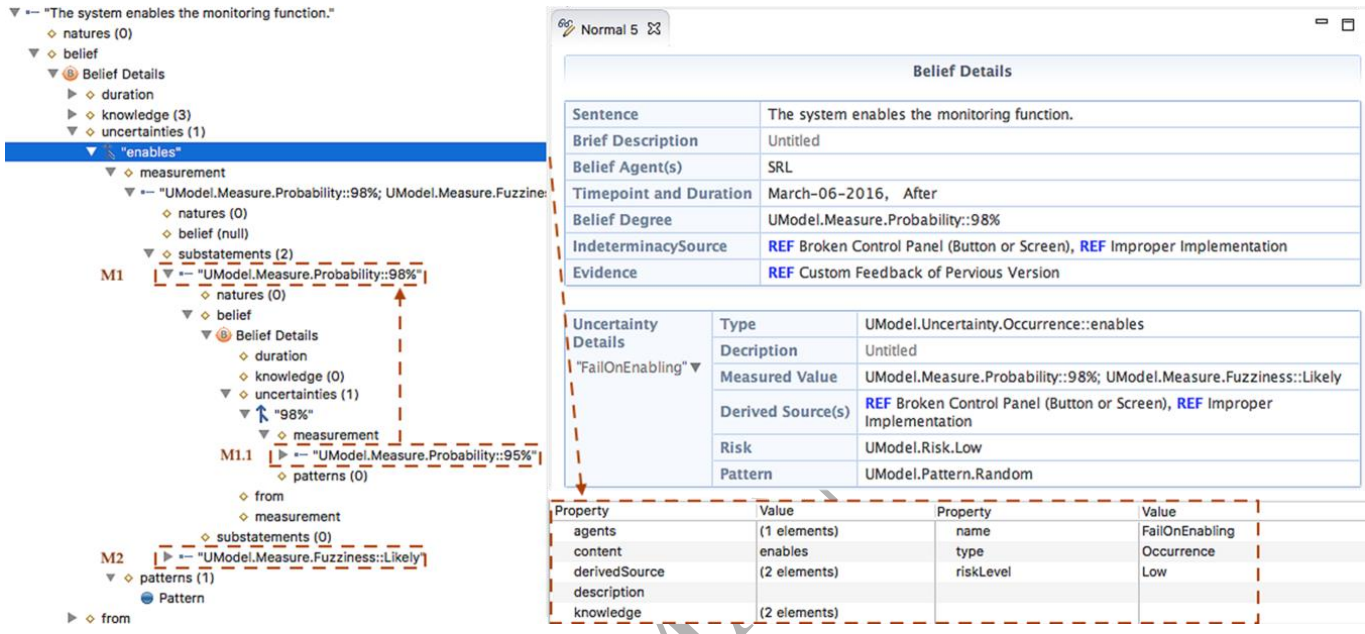


*A2 *Belief Agent* is formalized into elements shown in *A1*; the properties of *B1(D1) IndeterminacySource (Measurement)* is shown in the property window *B3 (D3)*; *C2* is the set of knowledge that are formalized as elements shown in *C1*; *E2* refers to a set of sentences indicated by *E1*; the belief sentences indicated by *F2* are formalized into elements shown in *F1*.

Figure 1. Specifying Belief Use Case Specification of *Monitor Windows and Doors*

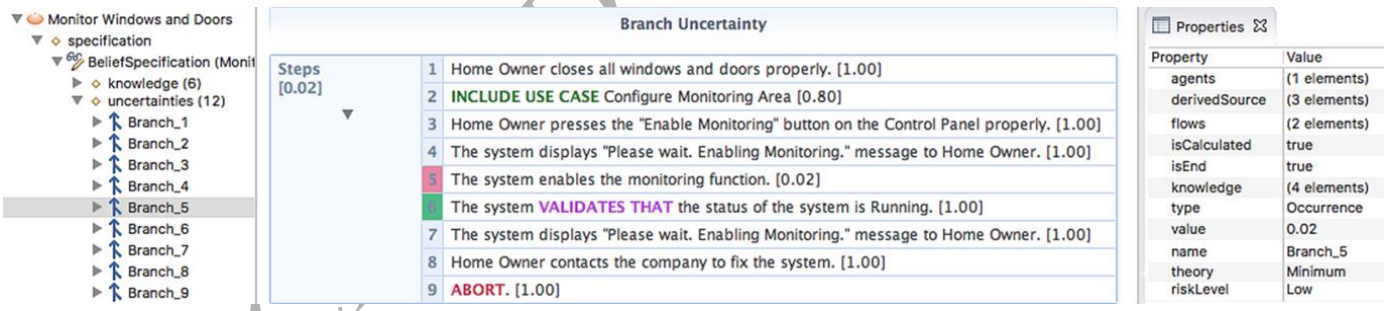
Since RUCM was initially proposed by Yue et al. [24] in 2009, multiple extensions have been proposed. A restricted test case modeling methodology was presented in [10] to generate executable test cases automatically. In [14], Wang et al. also presented another RUCM-based approach to generate

test cases from use case models automatically. Both of these approaches have been evaluated using real industrial case studies. Wu et al. [25] extended RUCM for specifying safety requirements in the domain of safety-critical systems. The authors of [26] presented an approach for facilitating feature-oriented requirements validation in the context of automotive systems, where RUCM was used to specify system features.



M1 and *M2* are the two formalized measurement statements. *M1.1* is a measurement statement defined for *M1*.

Figure 2. Specifying the Belief Sentence and an Associated Uncertainty (*NLUncertainty*) of *Normal* step *5* and Measurement Statement



*A step highlighted with Green (or Red) means that the condition is validated to be true (or the uncertainty does not occur).

Figure 3. An Example of Generated *BranchUncertainty* (across *Normal* and *FailOnEnablingMonitoring*)

3 U-RUCM Templates and Restriction Rules

As noted in Section 2.3, the RUCM methodology has two key distinguishing features: specifying UCSs with the RUCM template, and applying the RUCM restrictions (including the keywords) to guide the way in which users use natural language to specify the control flows of UCSs. U-RUCM extends RUCM by proposing two U-RUCM templates, adapting the RUCM restriction rules, and introducing new ones.

3.1 Templates

One of the U-RUCM templates is for specifying BUCSs as shown in Table 1. The BUCS template

inherits the key heading fields of the RUCM template, such as *Use Case Name* and *Brief Description*. In addition, U-RUCM introduces six new fields to indicate: 1) who specifies the BUCS (*Belief Agent(s)*), 2) when it was specified and the length of time during which the belief agent(s) holds the belief (*Time Point and Duration*), 3) the degree to which the belief agent(s) believes the specification (*Belief Degree*), 4) a set of indeterminacy sources that resulted in the uncertainties of the BUCS (*Indeterminacy Source(s)*), 5) evidence used to support this BUCS and its belief and uncertainty elements (*Evidence*), and 6) the precondition of the BUCS on which the belief and uncertainties are founded (*Belief Precondition*).

Table 1. The U-RUCM Templates for Belief Use Case Specifications (BUCSs) and Uncertainties

The template for specifying a BUCS

Use Case Name	The name of the use case. It usually starts with a verb.	
Brief Description	Summarizes the use case in a short paragraph.	
Primary Actor	The actor who initiates the use case.	
Secondary Actor(s)	Other actors the system relies on to accomplish the services of the use case.	
Dependency	Include and extend relationships to other use cases.	
Generalization	Generalization relationships to other use cases.	
Belief Agent(s)	One or more agents who hold belief about this BUCS.	
Time Point and Duration	The time point when the BUCS is specified and the duration in which the belief agent(s)'s belief on the BUCS holds.	
Belief Degree	The degree to which the belief agent(s) believe the BUCS.	
Indeterminacy Source(s)	The set of indeterminacy sources related to the BUCS (REF is used).	
Evidence	Evidence to support the BUCS, and its contained belief and uncertainty elements (REF is used).	
Belief Precondition	Belief agent(s)' belief on the precondition of the BUCS, which describes what should be true before the use case is executed.	
Belief Basic Flow (*Belief degree*)	Specifies the main successful path, also called "happy path".	
	Steps (numbered)	A set of ordered **belief sentence**s.
	Belief Postcondition	Belief agent(s)' belief on what should be true after the basic flow executes.
Belief Specific Alternative Flow (*Belief degree*)	Applies to one specific step of the reference flow.	
	URFS	The reference flow step where the belief agent(s) believe there are uncertainties.
	Alternative Step	An alternative to the reference flow step.
	Steps (numbered)	A set of ordered belief sentences.
	Belief Postcondition	Belief agent(s)' belief on what should be true after the specific alternative flow executes.
Belief Bounded Alternative Flow (*Belief degree*)	Applies to more than one step of the reference flow, but not all of them.	
	URFS	A list of reference flow steps where the belief agent(s) believe there are uncertainties.
	Alternative Steps	A set of alternatives to the reference flow steps.
	Steps (numbered)	A set of ordered **belief sentences**.
	Belief Postcondition	Belief agent(s)' belief on what should be true after the bounded alternative flow executes.
Belief Global Alternative Flow (*Belief degree*)	Applies to all the steps of the reference flow.	
	Belief Branching Condition	Belief agent(s)' belief on the condition, which describes what should be true when branching from any of the steps of the reference flow.
	Steps (numbered)	The set of ordered **beliefs sentences.**
	Belief Postcondition	Belief agent(s)' belief on what should be true after the global flow executes.

The template of specifying a belief sentence (BS) with one and more Uncertainties

Time Point and Duration	The time point when the BS is specified and the duration in which the belief agent(s)'s belief on the BS holds.
Belief Degree	The degree to which the belief agent(s) believe the BS.
Indeterminacy Source(s)	The set of indeterminacy sources related to the BS (REF is used).
Evidence	Evidence to support the BS, and its contained belief and uncertainty elements (REF is used).

Uncertainty Details	Specifies the details of the Uncertainty in the BS.	
	Type	The type of this uncertainty (Occurrence/Content/Time/Environment/GeographicalLocation)
	Indeterminacy Source(s)	The set of indeterminacy sources related to this Uncertainty (REF is used).
	Measure Value	The measurement of this uncertainty.
	Risk	The possible risk led by this uncertainty.
	Pattern	The pattern of the occurrence of this uncertainty

As discussed in Section 2, each RUCM UCS has one and only one basic flow and, optionally, three types of alternative flows. U-RUCM extends each type of event flows by 1) introducing a *belief degree*, which measures the degree to which the belief agent(s) believes a specific flow, 2) introducing a new keyword, *URFS* (Uncertain Reference Flow Step(s)), from which an alternative flow of events branches out, 3) providing the capability to annotate sentences in steps of flows and postconditions with belief and uncertainty information, and 4) introducing the new concept of *alternative steps* to enable the specification of uncertainties for alternative steps across flows of events. Note that for the case of a global alternative flow, a condition for branching from any step in the flow, should be specified via the *Belief Branching Condition* field. In Section 4, we formally define each of these fields and concepts.

We have developed an editor for U-RUCM (Section 5). The running example shown in Figure 1 is specified with the U-RUCM editor. The use case has one basic flow (called *Normal*) and several alternative flows. The complete specification is provided in [27] for reference.

The other U-RUCM template is used for specifying the belief sentence (BS) with uncertainty as shown in Table 1. An example is shown in Figure 2. This template has fields for identifying indeterminacy sources and evidence, and for specifying uncertainty properties such as *Type*, *Pattern*, and *Measured Value*. More details are provided in Section 4. In addition, an example is shown in Figure 3, demonstrating uncertainties for a BUCS (i.e., BranchUncertainty) derived by the editor automatically.

3.2 Restriction Rules

Based on the 26 RUCM restriction rules (Section 2.3), we systematically checked the applicability of them in the context of *U-Model* and identified the correspondence of the U-RUCM restriction rules (UR1-UR25) and the RUCM restriction rules (R1-R26), as summarized in Table 2. Fifteen out of the 26 RUCM rules (e.g., R1, R2) are remained in the U-RUCM rule set. R3 is removed from the U-RUCM rule set since the application context of U-RUCM is expanded to cover complex systems such as CPSs and allowing actor-to-actor interactions also facilitates the explicit specification of uncertainty related to devices and subsystems of a system. R10 and R11 are relaxed and become UR9 and UR10. For instance, R10, i.e., *Don't use modal verbs (e.g., might)*, is relaxed by allowing the use of *might* and *may*. This is

because these two modal verbs are often used to express uncertainty in natural language and it can enable reasoning about uncertainty as demonstrated in [2, 3]. In addition, we extend some RUCM keywords by giving them additional uncertainty related semantics (UR16 to UR22). For instance, DO-UNTIL with uncertainty (UR22) represents an uncertain belief whether a condition is correctly specified for the exit iteration action, and thus uncertainty can only be addressed on the exit condition of the DO-UNTIL sentence.

Table 2. U-RUCM restriction rules corresponding to the RUCM restriction rules

U-RUCM			**RUCM**	
#	**Description**	**Mapping**	**#**	**Description**
UR1		Same	R1	The subject of a sentence in basic and alternative flows should be the system or an actor.
UR2		Same	R2	Describe the flow of events sequentially.
	U-RUCM allows to describe actor-to-actor interactions since a device or a sub-system of the system (e.g., CPSs) can be considered as an actor when specifying use cases for another device or subsystem of the same system. Such interactions include for example a subsystem's behaviors requires inputs from another subsystem by communicating with it.	Removed	R3	Actor-to-actor interactions are not allowed.
UR3		Same	R4	Describe one action per sentence. (Avoid compound predicates.)
UR4		Same	R5	Use present tense only.
UR5		Same	R6	Use active voice rather than passive voice.
UR6		Same	R7	Clearly describe the interaction between the system and actors without omitting its sender and receiver.
UR7		Same	R8	Use declarative sentence only. "Is the system idle?" is a non-declarative sentence.
UR8		Same	R9	Use words in a consistent way. Keep one term to describe one thing.
UR9	Modal verbs of *might* or *may* can be used to express the possible occurrence of an action.	Relaxed	R10	Don't use modal verbs (e.g., might)
UR10	Adverbs can be only used in a measurement statement measured with Likert scales, e.g., Very Likely.	Relaxed	R11	Avoid adverbs (e.g., very).
UR11		Same	R12	Use simple sentences only. A simple sentence must contain only one subject and one predicate.
UR12		Same	R13	Don't use negative adverb and adjective (e.g., hardly, never), but it is allowed to use not or no.
UR13		Same	R14	Don't use pronouns (e.g. he, this)
UR14		Same	R15	Don't use participle phrases as adverbial modifier. For example, the italic-font part of the sentence "ATM is idle, displaying a Welcome message", is a participle phrase.
UR15		Same	R16	Use "the system" to refer to the system under design consistently.
UR16	Uncertainty addressed on INCLUDE USE CASE can only be of the *occurrence* type for describing an uncertain belief on whether the included use case does proceed at this step.	Extended	R17	INCLUDE USE CASE
UR17	Uncertainty addressed on EXTENDED BY USE CASE can only be of the *occurrence* type for describing an uncertain belief on whether the extended use case does proceed at this step.	Extended	R18	EXTENDED BY USE CASE
UR18	Do not use RFS to refer to any sentence that has uncertainty. In addition, no alternative step must be contained for a RFS alternative flow.	Extended	R19	RFS
UR19	Uncertainty can only be addressed on the IF and/or ELSEIF condition of the IF-THEN-ELSE-ELSEIF-ENDIF sentence for describing an uncertain belief on whether the content of such a condition is correctly specified (the *content* type) or whether the condition is occurred (evaluated to be true) (the *occurrence* type).	Extended	R20	IF-THEN-ELSE-ELSEIF-ENDIF
UR20	Uncertainty addressed on MEANWHILE can only be of the *time* type for expressing an uncertain belief if two actions occur concurrently.	Extended	R21	MEANWHILE
UR21	Uncertainty addressed on keyword VALIDATES THAT can	Extended	R22	VALIDATES THAT

	only be of the *occurrence* type for describing an uncertain belief on whether the system functions the validation check correctly. Uncertainty can also be addressed on the condition for describing an uncertain belief on whether the condition is correctly specified with *content* type or whether the condition is occurred (evaluated to be true) with the *occurrence* type.			
UR22	Uncertainty can only be addressed on the exit condition of DO-UNTIL with the *content* type for describing an uncertain belief on whether the condition is correctly specified.	Extended	R23	DO-UNTIL
UR23		Same	R24	ABORT
UR24		Same	R25	RESUME STEP
UR25		Same	R26	Each basic flow and alternative flow should have their own postconditions.

Moreover, we introduced additional rules (i.e., UR26 to UR42 in Table 3) for explicitly capturing uncertainty-related information. First, U-RUCM defines a set of new rules for capturing necessary information (i.e., UR26 to UR29) and restraining input formats (i.e., UR30, UR32, UR34). Second, U-RUCM introduces two new keywords: REF (UR38) and URFS (UR39). REF is introduced for specifying associated indeterminacy sources and evidence of a BUCS, BS or uncertainty such that they can be referenced in multiple places within a BUCS. For example, the field *Indeterminacy Source(s)* lists all the defined indeterminacy sources of the specification, each of which is referenced to with REF (Figure 1, C1 and C2). Like the RUCM RFS keyword, the URFS keyword is introduced for associating an alternative flow to the steps in its reference flow. However, RFS is used for branching out from a reference flow step under a fully certain condition. For example, in Figure 1, the alternative flow *DetectIntrusion* uses RFS to indicate that it branches from step 10 of the basic flow (*Normal*). In contrast, URFS is provided to associate uncertainties across flows of events. For example, in the running example (Figure 1, E1 and E2), the URFS keyword is applied to *FailOnEnablingMonitoring* to show that, in an uncertain unknown condition, it is possible that the alternative sentences *A1 to A2* can "replace" steps 5 to 7 of the basic flow. Third, U-RUCM defines a set of constraints to help a user to input complete and valid uncertainty information (i.e., UR31, UR33, UR35 to UR37, UR40 to UR42). For example, for any URFS alternative flow, there should be at least one alternative sentence specified in the alternative flow, which "replaces" the referenced steps of the reference flows of events defined in the URFS statement (UR41).

We also want to clarify that for keywords with conditions, i.e., DO-UNTIL, VALIDATES THAT, and IF-THEN-ELSE-ELSEIF-ENDIF, if their conditions contain uncertainty with the *content* type, it implies that the belief agent has uncertain belief on whether such a condition is correctly specified. For keywords VALIDATES THAT and IF-THEN-ELSE-ELSEIF-ENDIF, if their conditions contain uncertainty with the *occurrence* type, it implies that the belief agent has uncertain belief on whether such a condition is occurred (evaluated to be true). But for DO-UNTIL, it is not meaningful to introduce occurrence type of uncertainty for its exit condition because the condition has to occur to

avoid any infinite loop. Note that whether or not there is an uncertainty in the conditions of these three keywords has no impact on the logic of the execution of their action steps but the measurements of the specified uncertainty on the conditions indicate belief agents' confidence on the condition contents (for *content* type of uncertainty) and/or the possibility of the occurrences of the conditions (for *occurrence* type of uncertainty).

Table 3. 17 U-RUCM restriction rules that are newly added

#	Description	#	Description
UR26	BeliefAgent, IndeterminacySource and Evidence must have its unique name specified.	UR27	IndeterminacySource must have its nature specified.
UR28	BeliefAgent of a BUCS must be specified.	UR29	Timepoint and Duration of a BUCS must be specified.
UR30	Timepoint and Duration should be specified with the format of "[timepoint], [Duration]".	UR31	Timepoint of a BS in a BUCS must be no later than the time point of the BUCS. The duration of a BS in a BUCS must be no less than the duration of the BUCS.
UR32	Measurement should be specified with its measure with the format of "[Measure]::[Measurement]" .	UR33	Measure of each measurement owned by the same BUCS/BS/Uncertainty must be different.
UR34	Uncertainty should be specified with its type with the format of "[type]::[PoS]".	UR35	Uncertainty in a BS can only be addressed on its content (e.g., the predicator of the sentence)
UR36	Uncertainty cannot be addressed on a measurement statement for describing an uncertainty of the measurement statement.	UR37	Indeterminacy source (evidence) referred in a BS in a BUCS must belong to the indeterminacy source (evidence) set referred by the BUCS.
UR38	REF	UR39	URFS
UR40	Do not use URFS to refer any sentence that no uncertainty is specified.	UR41	For a URFS alternative flow, at least one alternative step must be specified.
UR42	No uncertainty should be addressed on the keywords of IF-THEN-ELSE-ELSEIF-ENDIF, DO-UNTIL, ABORT, RESUME STEP, RFS and URFS.		

4 U-RUCM SYNTACTIC FORMALIZATION

Since requirements are specified by requirements engineers based on their knowledge and experience, requirements specifications naturally contain information subject to the requirements engineers. *U-Model* was proposed from the subjective perspective and when integrating it with use case modeling, it fits well for expressing subjective characteristics of use case specifications, and most importantly for representing subjective uncertainties in the specifications, i.e., about what the requirements engineers are aware of that they lack prefect knowledge. Therefore, the syntactic formalization of U-RUCM is realized via an integration of an extended version of UCMeta and *U-Model*. The full formalization is captured in a distinct metamodel, called *BeliefUCMeta*.

The integrated metamodel (*BeliefUCMeta*) serves as an intermediate metamodel for enabling various types of automation via model transformations, e.g., automated reasoning of uncertainty and generation of other downstream artifacts such as analysis models and even test cases. Such an intermediate metamodel is very important as it enables the separation of the transformation from textual use case models specified with U-RUCM to targeting models (e.g., UML analysis models or test cases) into two parts: the textual use case models to the instances of BeliefUCMeta and from them to the targeting models. The first part of the transformation can be reused for generating various types of

targeting models. We have made the implementation of BeliefUCMeta available[4] and any interested readers can use it for their own purposes.

Note that the semantics formalization at the sentence level is not considered as a contribution of this paper since UCMeta (corresponding to RUCM) covers it already.

4.1 Relationships of BeliefUCMeta with UCMeta and U-Model

As shown in Figure 4, *BeliefUCMeta* imports elements from *U-Model* and UCMeta, which formalizes RUCM. UCMeta is composed four packages: *UCTemplate* (corresponding to the RUCM template and formalizing template fields such as use case name, flows of events), *SentenceStructure* (formalizing sentence constructs such as *noun phrase*, *subject*), *SentenceSemantics* (formalizing different types of simple sentence patterns such as *subject-verb-direct object*) and *SentencePattern* (formalizing semantics functions of a use case model such as *Validation* for describing that the system validates a request and data).

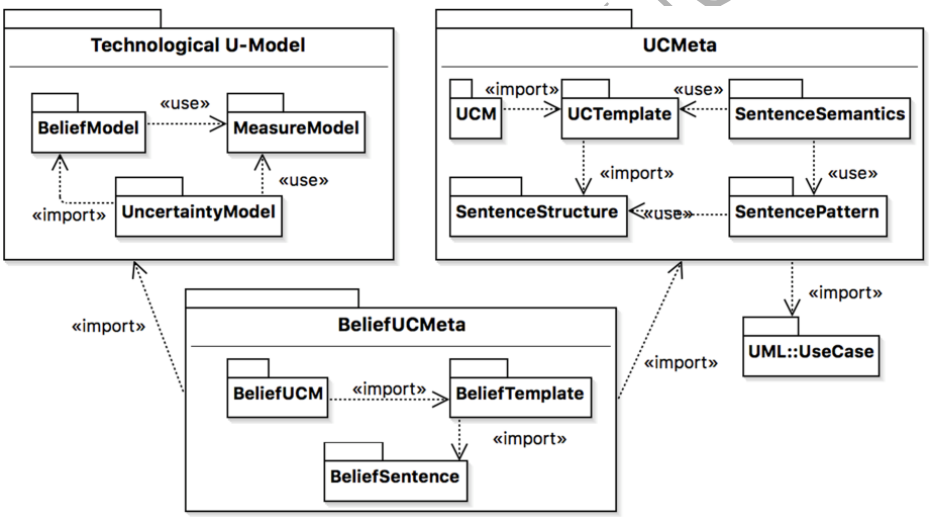


Figure 4 Relationships between BeliefUCMeta with UCMeta and Technological U-Model Metamodel

Concepts of *U-Model* (Section 2.1) are divided into three groups and implemented as a metamodel, which is named as the Technological *U-Model* metamodel to distinguish itself from the *U-Model* conceptual model. BeliefUCMeta is composed of three packages, as shown in Figure 4: BeliefUCM, *BeliefTemplate* and *BeliefSentence*. Since BeliefUCMeta imports UCMeta, BeliefUCMeta can naturally benefit from the existing capability of UCMeta for automatically formalizing, by relying on natural language processing techniques, sentences into instances of metaclasses of packages SentenceSemantics, SentencesStructure and SentencePattern [8]. The complete lists of metaclasses of UCMeta, Technological *U-Model* and BeliefUCMeta are provided in [27] for reference. In addition, to specialize concepts of UCMeta for BeliefUCMeta, we also defined a set of constraints specified with

[4] http://zen-tools.com/rucm/metamodels/belief_ucmeta/index.html

OCL as shown in Table 4. Moreover, based on BeliefUCMeta, we specified newly introduced restriction rules with OCL as shown in Appendix B. Note that constraints defined in this section were implemented in the U-RUCM tool for automatically generating correct instances of BeliefUCMeta. They are different from the constraints defined in Appendix B for validating inputs of a user when using the U-RUCM tool.

4.2 Belief Use Case Model, Element, and Classifier

BUElement, which specializes *UseCaseElement* of UCMeta, is the root of all the other elements of *BeliefUCMeta* (Figure 5). In other words, all the other metaclasses of *BeliefUCMeta* specialize *BUElement*.

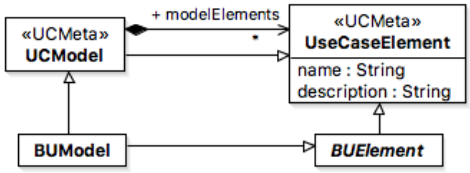


Figure 5. Root Elements of BeliefUCMeta

BeliefClassifier (Figure 6) is an abstract metaclass, introduced in U-RUCM to support the classification of a set of BUCS elements (e.g., *BeliefPrecondition*, *BeliefFlowOfEvents*), to which belief and uncertainty information can be attached. *BeliefClassifier* specializes *BeliefStatement* of *U-Model* (Appendix A.1), through which a belief classifier is associated with *Uncertainty*. We would like to point it out that "Belief" is the core element in *U-Model* for presenting subjective characteristics of belief statements and expressing uncertainty, and therefore we extended relevant RUCM elements with "*belief*" (i.e., *BeliefClassifier* and its subtypes, Figure 6).

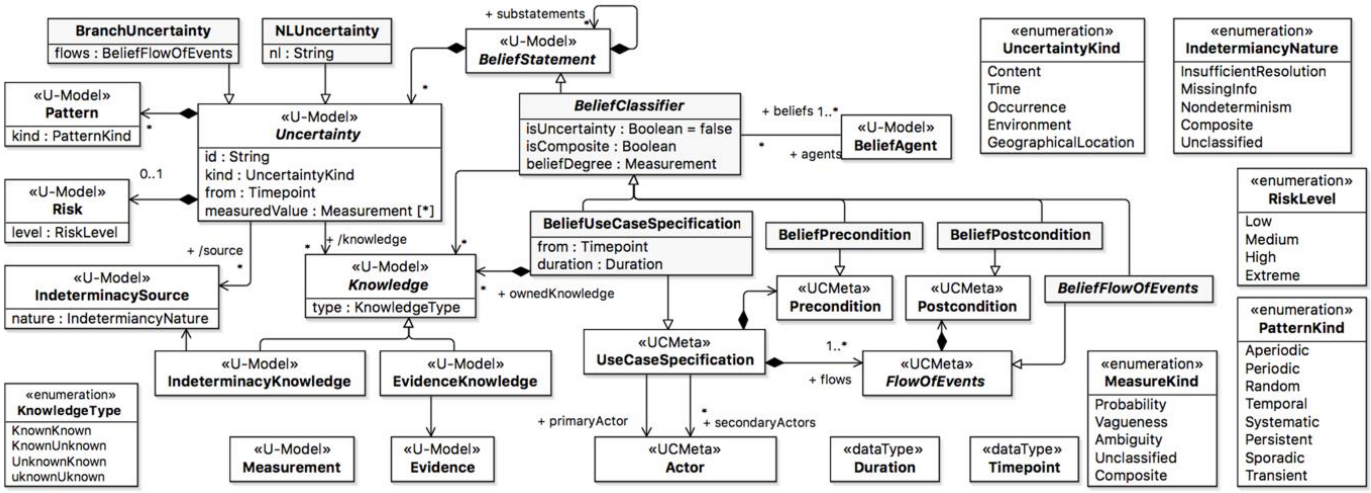
BeliefClassifier has three attributes: *isUncertainty : Boolean* (indicating if a belief classifier is associated to any known or unknown uncertainty), *isComposite : Boolean* (indicating if a belief classifier can be decomposed into other belief elements), and *beliefDegree : Measurement* (formalizing the degree to which a belief agent believes in a belief classifier). More details on measurements are provided in Section 4.6. A belief classifier may be associated to one or more *BeliefAgent*s, which is defined, in *U-Model*, as an individual, a community of individuals sharing the same set of beliefs, or technology, such as a software system, with built-in beliefs (Appendix A.1). For example, as shown in Figure 1, the belief agent of the BUCS is *SRL*, the Simula team who developed the case study.

A belief classifier may be associated with *IndeterminacyKnowledge* and/or *EvidenceKnowledge*. *IndeterminacyKnowledge* is defined, in *U-Model*, to describe an objective relationship between an *IndeterminacySource* and the awareness that the *BeliefAgent* has of that source (Appendix A.1) and

whereas *EvidenceKnowledge* in *U-Model* captures an objective relationship between an *IndeterminacySource* and *Evidence* (Appendix A.1). As the subtypes of metaclass *Knowledge*, *IndeterminacyKnowledge* and *EvidenceKnowledge* inherit the attribute of *Knowledge*, i.e., *type* typed with enumeration *KnowledgeType* with four literals: *KnownKnown*, *KnownUnknown*, *UnknownKnown*, and *UnknownUnknown*, definitions of which are provided in Appendix A.1 for reference. The two types of knowledge are respectively associated with *IndeterminacySource* and *Evidence*, which are discussed in the next sections with details.

Table 4. Constraints defined on BeliefUCMeta

ID	Name	Constraints in Object Constraint Language (OCL)
Con1	The belief degree of a belief classifier is 100% if its *isUncertainty* attribute takes value of *False*.	**context** BeliefClassifier **inv**: (**not self**.isUncertainty) **implies** (**self**.beliefDegree = **null or** (**self**.beliefDegree <> **null and self**.beliefDegree.value = 1.0))
Con2	The precondition of a BUCS should be a *BeliefPrecondition*.	**context** BeliefUseCaseSpecification **inv**: **self**.precondition.**oclIsKindOf**(BeliefPrecondition)
Con3	All *FlowOfEvents* of a BUCS should be *BeliefFlowOfEvents*.	**context** BeliefUseCaseSpecification **inv**: **self**.flows->forAll(f:UCMeta::UCTemplate::FlowOfEvents| f.**oclIsKindOf**(BeliefFlowOfEvents))
Con4	All the uncertainties owned by a BUCS are *BranchUncertainty*.	**context** BeliefUseCaseSpecification **inv**: **self**.uncertainty->forAll(u:Uncertainty|u.**oclIsKindOf**(BranchUncertainty))
Con5	All sentences in belief flows of events are *BeliefSentences*.	**context** BeliefFlowOfEvents **inv**: **self**.steps->forAll(bs:UCMeta::UCTemplate::Sentence|bs.**oclIsKindOf**(BeliefSentence)) **and** (**self**.**oclIsKindOf**(AlternativeFlow) **implies self**.rfs->forAll(s:UCMeta::UCTemplate::Sentence|s.**oclIsKindOf**(BeliefSentence)))
Con6	The condition sentence of a *BeliefGlobalAlt* must not be null and it must be a *BeliefSentence*.	**context** BeliefGlobalAlt **inv**: **self**.condition->size()>0 **and self**.condition.**oclIsKindOf**(BeliefSentence)
Con7	All the uncertainties associated to a belief sentence are of the *NLUncertainty* type.	**context** BeliefSentence **inv**: **self**.uncertainty->size()>0 **implies self**.uncertainty->forAll(u:Uncertainty|u.**oclIsKindOf**(NLUncertainty))
Con8	All *altSteps* of a BeliefAlternativeFlow are alternative sentences (with *isAlternative* being *true*) and *urfs* of *Sentence* is a subset of *urfs* of the *BeliefAlternativeFlow*	**context** BeliefAlternativeFlow **inv**: **self**.altSteps->size()>0 **implies self**.altSteps->forAll(s:UCMeta::UCTemplate::Sentence|s.**oclAsType**(BeliefSentence).isAlternative **and self**.urfs->includesAll(s.**oclAsType**(BeliefSentence).urfs))



*UCMeta metaclasses are indicated with «UCMeta»; U-Model elements are highlighted with «U-Model»; the other metaclasses are newly proposed in U-RUCM. Constraints specified on this part of the metamodel are provided in Table 4.

Figure 6. The core of BeliefUCMeta

4.3 Belief Use Case Specification

As shown in Figure 6, *BeliefUseCaseSpecification* specializes *BeliefClassifier* and *UseCaseSpecification* of UCMeta. Same as for RUCM specifications, a U-RUCM specification is composed of a set of flows of events, precondition, each flow of event is composed of one postcondition, and each specification is associated to one primary actor and optionally associated to one or more secondary actors. In general, *BeliefUseCaseSpecification* is a concrete specification of a use case specified by a *BeliefAgent*. It includes information about the agent's confidence that the use case will execute as specified. For example, as shown in Figure 1, the *Monitor Windows and Doors* use case of the *SafeHome* case study is specified as a *BeliefUseCaseSpecification* (a subtype of *BeliefClassifier*), with its belief degree specified as 80%. A BUCS has two attributes: *from* and *duration*, which indicate when it was specified and the validity period of the belief. For example, for the running example, the use case was specified on March 3rd of 2016 and the belief is valid since after. Notice that these two attributes are inherited from *BeliefStatement* of *U-Model* (Appendix A.1).

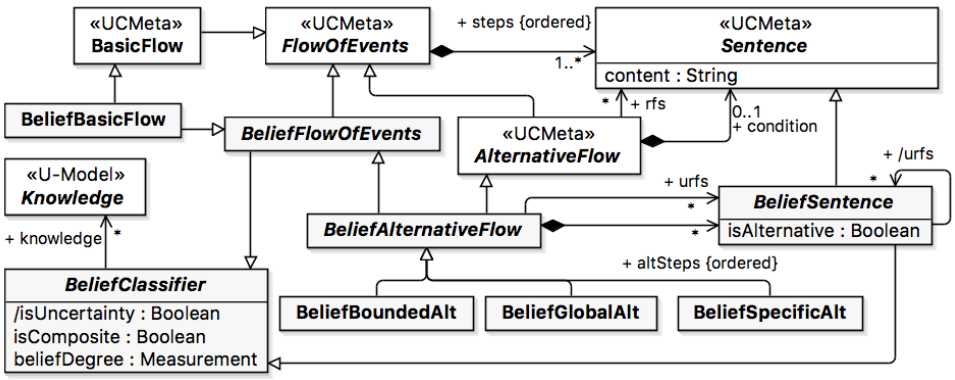
The BUCS template of U-RUCM (Table 1) conforms to *BeliefUCMeta*. An instance of *BeliefUseCaseSpecification* is created for each BUCS and serves as the container of other belief-related elements (e.g., *BeliefPrecondition*, *BeliefFlowOfEvents*, *BeliefSentence*). Different from a traditional UCS of RUCM, a BUCS includes specifications of *IndeterminacySource* and *Evidence* relevant to the use case. For example, as shown in Figure 1, the fields of *Indeterminacy Source (s)* and *Evidence* of the U-RUCM editor should be used for listing all indeterminacy sources and evidence relevant to the *Monitor Windows and Doors* use case. Notice that, the *REF* keyword should be used to refer to existing instances of *IndeterminacySource* and *Evidence* metaclasses of U-RUCM, definitions of which are from *U-Model* and provided in Appendix A for references. For example, all the indeterminacy sources defined for the

belief use case model are displayed on the left side of Figure 1, starting with "INT". Furthermore, in the property window (e.g., the *B2* section of Figure 1) of an indeterminacy source, one can specify its name and nature (e.g., *Nondeterminism* of *IndeterminacyNature*). The attribute *nature* of *IndeterminacySource* is typed with *IndeterminacyNature* defined by five enumeration literals representing five possible indeterminacy sources. Notice that indeterminacy sources owned by BUCSs can be referenced by other belief elements such as *Uncertainty*. Such references are formalized as instances of *IndeterminacyKnowledge* and *EvidenceKnowledge* of *BeliefUCMeta* (Figure 6) such as C1 and C2 shown in Figure 1.

4.4 Belief Flow of Events

BeliefFlowOfEvents is a subtype of *BeliefClassifier* and also extends *FlowOfEvents* of UCMeta (Figure 7). Hence, it inherits *BasicFlow* and the three types of alternative flows: *Specific*, *Bounded* and *Global* of RUCM and therefore UCMeta, as shown in Figure 7. A belief flow of events is composed of a set of sentences, which are all belief sentences (Section 4.5), as enforced by the Con5 constraint (Table 4). *BeliefFlowOfEvents*, as a specific type of *BeliefClassifier*, has a derived association to *Knowledge* (Figure 7), through which a belief flow of events can be associated with indeterminacy sources and evidence if needed.

Specific and bounded alternative flows can be differentiated based on whether RFS or URFS is used to refer to one or more steps of a reference flow. RFS is used only when the branching condition is fully clear to the belief agent (UR18). In other words, the belief sentence corresponding to the branching condition has its belief degree specified at 100% and its *isUncertainty* attribute specified as *False* (the Con1 constraint, Table 4). For example, the *DetectIntrusion* flow in Figure 1 branches out from step 10 of the basic flow under the condition that "the windows and doors are open". The belief degree to the sentence (step *1* of *DetectIntrusion*) corresponding to this condition is 100%.



*Constraints specified on this part of the metamodel are provided in Table 4.

Figure 7 BeliefFlowOfEvents of BeliefUCMeta

In contrast, URFS is used when the belief agent is not fully confident about a particular system behavior or condition (represented by one or more steps in a flow). In principle, an URFS alternative

flow should always be linked to one or more indeterminacy sources. If such indeterminacy sources are known, U-RUCM provides a way to specify them (see Section 4.3). For example, the *FailOnEnablingMonitoring* flow is defined as that the belief agent is not fully confident that "the system enables the monitoring function" (step 5 of the basic flow, Figure 1) will actually occur. URFS (instead of RFS) is used in this context because the behavior is branched out due to uncertainty in one or more of the referred sentences. For example, since the condition to evaluate whether the system invokes function to enable monitoring is uncertain, as shown in step 5 of the basic flow (Figure 1), we used URFS for branching out from this step.

For global alternative flows, there is no need to use RFS and URFS. However, a global condition must be defined for a global alternative flow (Figure 7), which is a constraint defined by RUCM and also applies to U-RUCM. For example, as shown in Figure 1, *CallPolice* is a global alternative flow, which defines the global condition: "The Alarm has been triggered for more than 5 minutes." Since the condition is a belief sentence, one can associate uncertainties with it if needed.

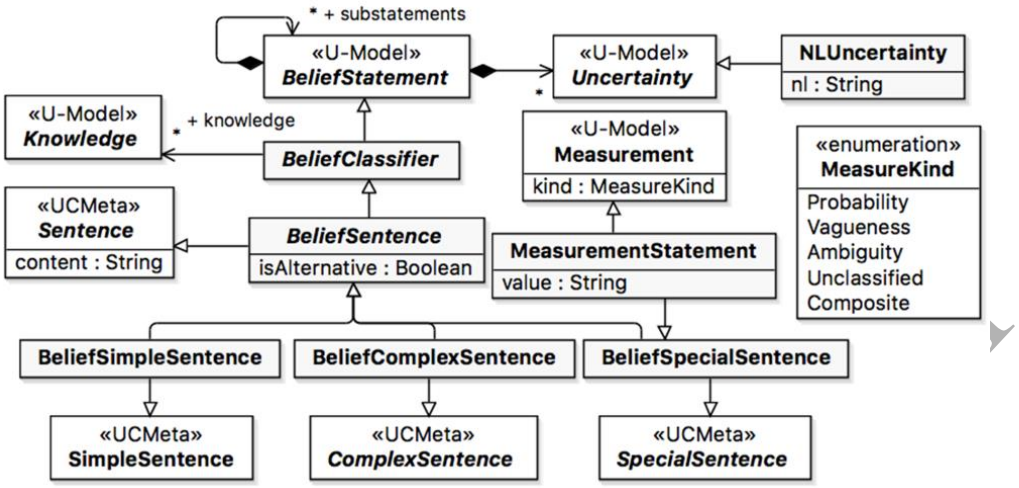
In summary, U-RUCM provides four different ways of specifying alternative flows: *Bounded* with RFS, *Specific* with RFS, *Bounded* with URFS and *Specific* with URFS. Global alternative flows cannot be combined with RFS and therefore URFS, as we discussed above. We also defined corresponding restriction rules for specifying the flows. For instance, the UR41 (i.e., for any URFS alternative flow, there should be at least one alternative sentence specified in the alternative flow, which "replaces" the referenced steps of the reference flows of events defined in the URFS statement) can be specified with OCL constraint as shown UR41 in Appendix B. Note that alternative sentences (which are ordered as sequential steps) are defined at the beginning of an URFS alternative flow, followed by a sequence of regular belief sentences.

4.5 Belief Sentence

All sentences in a BUCS are belief sentences, which are formalized as *BeliefSentence* elements (specializing the *Sentence* concept of UCMeta). Since *BeliefSentence* is a subtype of *BeliefClassifier*, belief sentences inherit all the attributes of *BeliefClassifier* (e.g., *beliefDegree*, *isUncertainty*, Figure 6), which distinguish themselves from regular RUCM sentences. In RUCM and UCMeta, sentences are classified into *simple, complex* and *special* sentences. Consequently, U-RUCM and *BeliefUCMeta* classify belief sentences into *BeliefSimpleSentence*, *BeliefComplexSentence* and *BeliefSpecialSentence* (Figure 8).

A belief sentence can be associated with *Uncertainty* via *BeliefClassifier* and *BeliefStatement* and with *IndeterminacySource* and *Evidence* via *Knowledge* and *BeliefClassifier* as shown in Figure 8. A belief sentence uncertainty must be a *NLUncertainty* (the Con7 constraint, Table 4) and the *String* value of its *nl* attribute should be part of the content of the belief sentence (UR35 and its OCL constraint in

Appendix B). More details about *NLUncertainty* are discussed in Section 4.6.



*Constraints specified on this part of the metamodel are provided in Table 4.

Figure 8. *BeliefSentence* of BeliefUCMeta

A belief simple sentence is an atomic belief statement, which, from the sentence structure perspective, is composed of only one subject and one predicate. Belief simple sentences can appear in action steps of flows, preconditions, postconditions, and other fields of a BUCS. Belief complex sentences are sentences with the following RUCM keywords applied: IF-THEN-ELSE-ELSEIF-THEN-ENDIF for conditions, DO-UNTIL for iterations, MEANWHILE for concurrency, and VALIDATES THAT for validation. A complex sentence can consist of one or more belief simple sentences. Belief special sentences are sentences involving keywords RESUME STEP, ABORT, RFS, EXTENDED BY, INCLUDE and URFS. Notice that special sentences involving keywords URFS are newly introduced in U-RUCM, and all the other special sentences are inherited from RUCM. It is important to point it out that U-RUCM benefits from the existing capability of UCMeta (which formalizes RUCM) for providing the formalization of the sentence structures, sentence patterns and sentence semantics (Section 4.1). U-RUCM can also benefit from the automated solution (i.e., aToucan [8]) of formalizing natural language sentences into instances of the metaclasses of these UCMeta packages. Doing so provides opportunities for automatically analyzing formalized belief use case models and transforming them into other artifacts when needed. For example, one possibility is to transform belief use case models into UML models specified in *UncerTum* [16] for facilitating MBT. However, providing such capability is beyond the scope of this paper.

In U-RUCM, we also introduce alternative sentences, which are formalized as the *isAlternative* attribute of *BeliefSentence* (Figure 8). From the natural language perspective, alternative sentences have no difference with other belief sentences. The only difference is that alternative sentences can only appear in URFS alternative flows as action steps (Section 4.4). In the current implementation of the U-

RUCM editor, alternative sentences (e.g., *A1* and *A2* of *FailOnEnablingMonitoring*, Figure 1) are denoted with A1, A2, etc. An alternative sentence can be any type of belief sentences: *simple*, *complex* or *special*, as shown in Figure 8.

We also define, in *U-Model*, *MeasurementStatement*, a special type of *BeliefSpecialSentence* (Figure 8). For example, as shown in Figure 2, the *Measured Value* field of the uncertainty specifies two measurement statements, i.e., *UModel.Measure.Probability::2%* and *UModel.Measure.Fuzziness::Likely*. Since it is a special type of belief special sentences, a measurement statement can be associated with all the belief sentence properties (including derived ones) such as *Uncertainty*. Please refer to Section 4.7 for detailed discussions of measurement statements, measurements and measures.

4.6 Uncertainty

Uncertainty is defined, in *U-Model*, for "representing a state of affairs whereby a belief agent does not have full confidence in a belief that it holds" (Appendix A.1). We adopt the definition of uncertainty from *U-Model* to U-RUCM and associate it with *BeliefClassifier*, as shown in Figure 6.

In *U-Model*, uncertainties are classified into five different types, which are inherited by U-RUCM and formalized as the five literals of enumeration *UncertaintyKind* in *BeliefUCMeta* (Figure 6): *Content*, *Time*, *Occurrence*, *Environment,* and *GeographicalLocation*. Definitions of these uncertainty types are provided in Appendix A.2 for reference. For example, the type of the uncertainty described in Figure 2 is *Occurrence* as indicated by the content in the field of *Type* of the editor.

Each uncertainty has a time point describing when the uncertainty is initialized. This attribute is formalized as an attribute of metaclass *Uncertainty* of *BeliefUCMeta* (Figure 6). For example, the uncertainty in Figure 2 was specified in *March-03-2016* and is active since then. An uncertainty can be optionally associated with *Risk*. Currently, in *U-Model* and therefore in U-RUCM, we define four risk levels: *Low*, *Medium*, *High* and *Extreme* (enumeration *RiskLevel*, Figure 6). For example, the uncertainty in Figure 2 is of low risk. To further characterize an uncertainty, it can be optionally (if information available) associated with one or more patterns (e.g., *Aperiodic*, *Sporadic*), which are formalized as enumeration *PatternKind* in *BeliefUCMeta* (Figure 6). For the uncertainty described in Figure 2, the belief agent is not aware of in which pattern the uncertainty appears, and therefore it is not specified. An uncertainty can be measured (i.e., *measuredValue: Measurement*, Figure 6) in different ways (i.e., *MeasureKind*, Figure 6). Details are described in Section 4.7.

In U-RUCM, we classify uncertainties into two basic types: *NLUncertainty* and *BranchUncertainty* (Figure 6). *NLUncertainty*(s) are defined at the level of belief sentences. Branch uncertainties can be derived automatically from flows of events and are owned by BUCSs (Con4 of Table 4). In addition to these two categories, in U-RUCM, uncertainties in flows of events can also be captured via URFS and

alternative flows (Section 4.4).

4.6.1 Uncertainty in Belief Sentences (NLUncertainty)

An *NLUncertainty* refers to a Part of Speech (PoS) (e.g., noun, verb) of a belief sentence, about which a belief agent lacks confidence. This is enabled by the *nl* attribute of *NLUncertainty* (Figure 8). For example, one instance of *NLUncertainty* in step 5 of the basic flow (Figure 2) shows that the belief agent is 98% confident about the occurrence of the event. The uncertainty is associated with "*enables*", which is the predicator verb of the belief sentence. Notice that the *nl* attribute is typed with *String*. However, if automated solutions can always be proposed to parse a belief statement and automatically associate an *NLUncertainty* (through specified *nl* information) with constructs of a belief sentence (e.g., subjects, predicators, objects) if needed. As we discussed in Section 4.5, UCMeta and aToucan provide such a capability, from which U-RUCM can benefit.

An *NLUncertainty* in a belief sentence can be optionally associated with one or more indeterminacy sources owned by the BUCS, the container of all belief sentences and therefore uncertainties (Figure 6). Doing so helps to provide additional information describing situations whereby the information required to ascertain the validity of the belief sentence is indeterminate (Appendix A.1). For example, as shown in Figure 2, the uncertainty is associated with two indeterminacy sources, implying that the belief agent believes that there is a 98% probability (not 100%) that the system enables the monitoring function and therefore 2% probability that the system does not enable monitoring under unknown conditions due to the broken control panel or improper implementation of the monitoring functionality. Furthermore, the predefined indeterminacy source categories (e.g., *MissingInfo*, *Non-determinism*, defined in Appendix A.1), formalized as the literals of enumeration *IndeterminacyNature* (Figure 6) provides more information about indeterminacy sources and therefore uncertainties. For example, the indeterminacy source of *Broken Control Panel (Button or Screen)* is of a *Nondeterminism* indeterminacy nature (as shown in the left bottom corner of Figure 1), implying that the phenomenon of the control panel being broken is either practically or inherently non-deterministic.

It is also worth mentioning that *U-Model* allows one to specify uncertainties in measurement statements, which is enabled by the fact that a measurement statement is a belief sentence and therefore it can have uncertainties specified. We, however, in *U-RUCM*, enforce that when specifying such an uncertainty, its *nl* attribute should be part of the value or the whole of it (UR35 and its OCL constraint in Appendix B). Notice that a measurement statement is composed of two parts: measure and value (Section 4.7). Such an uncertainty should be a *Content* type of uncertainty. For the example given in Figure 2, if an uncertainty is associated with the measurement statement (i.e., *UModel.Measure.Probability::98%*), *nl* of the uncertainty should be *98%*.

4.6.2 Branch Uncertainty

A set of branches can be derived from a BUCS, systematically by following different strategies. For example, in [10], we defined three test coverage strategies: all branch coverage strategy (covering all branches), all condition coverage strategy (covering all conditions of all branches at least once), and loop coverage strategy (ensuring that each loop (DO-UNTIL) is exercised exactly one, none, and x number of times, where x can be specified beforehand). We adopt these three coverage strategies and use them to generate branches systematically from U-RUCM models. Each of these derived branches represents a straight path from the precondition of the specification all the way to a postcondition of a flow of events. One example of such a branch is provided in Figure 3, which is automatically generated from the use case specification provided in Figure 1.

The occurrence of a particular path might be uncertain, which therefore forms a branch uncertainty. Such an uncertainty is an instance of *BranchUncertainty* (Figure 6) with the *UncertaintyKind::Occurrence* kind. Since such branches can be automatically generated, measurements of the branch uncertainties can be automatically calculated when needed, if and only if uncertainties of the belief sentences of the specifications are specified. For example, as shown in Figure 3, the branch takes the path of branching from step 5 and executing the *FailOnEnablingMonitoring* alternative flow and finishing at its last step. Since the chance of the system does not enable the monitoring is 1-0.98=0.02 as indicated at step 5 of the branch, the overall branch uncertainty can then be defined as 0.02 if we follow the simple strategy of taking the lowest value of the belief degrees of the belief sentences as the branch uncertainty measurement. We acknowledge that there are many alternatives regarding how to systematically obtain branch uncertainty measurements. Users can also manually define such measurements if they want. However, studying such strategies is out of the scope of this paper.

4.7 Measurement

As discussed in Section 4.5, in U-RUCM, we define *MeasurementStatement* as a special type of *BeliefSpecialSentence* (Figure 8). *MeasurementStatement* also inherits *U-Model*'s *Measurement*; therefore, each measurement statement should be associated with a specific type of *Measure*s (Figure 8). A U-RUCM measurement can take different kinds of measures such as *Probability*, *Vagueness,* and *Ambiguity*, which are formalized as enumeration *MeasureKind* of *BeliefUCMeta* (Figure 6). For example, as shown in Figure 2, two measurements are specified for the uncertainty in the field of *Measured Value* as *UModel.Measure.Probability::98%* and *UModel.Measure.Fuzziness::Likely* indicating that the probability the occurrence of enabling monitoring is 98% if measured with *Probability* and it likely occurs if it is measured with *Fuzziness*. Note that *U-Model* defines a taxonomy of measures (Appendix A.3), which is integrated in U-RUCM as it is.

In U-RUCM, measurement statements can be specified for quantifying the belief degree of a belief classifier and quantifying uncertainty. Note that belief degrees and uncertainty measurements can be specified from a subjective opinion of a belief agent, based on real data, or the combination of both. The first case describes the situation that at the requirements level, where requirements engineers may specify belief degrees and uncertainty measurements based on their experience, knowledge, and even preferences, especially when there is no data available as a reference. The second case describes the situation that when sufficient data is available and can be used to specify belief degrees and uncertainty measurements. For instance, when data was collected based on previous operations (e.g., 100 times) of similar systems, and uncertainties occurred 60 times. In this case, a belief agent may directly formulate the measurement as "60% of probability" for quantifying the belief degree. The third case is about considering real data as additional information for backing up a subjective judgment on a measurement, which might result in for example "70% of probability" instead of "60% of probability".

One special case is to define uncertainties in measurement statements as the way how uncertainties are specified for other types of belief sentences. This is enabled because measurement statements are defined as a special type of belief sentence, as we discussed earlier. In other words, the belief agent can attach an uncertainty (e.g., with the measurement statement specified as: *UModel.Measure.Probability::95%*, *M1.1* as shown in Figure 2) to the value of a measurement statement (e.g., *98%*) to indicate that she/he is not fully confident about the measurement statement (e.g., *UModel.Measure.Probability::98%*).

Since different belief agents might have different views on belief degree and uncertainty measurements, *U-Model* enables this by specifying a belief degree (or uncertainty measurement) as a specific type of *BeliefClassifier*, which is associated with one or more *BeliefAgent*s as shown in Figure 6. Moreover, *U-Model* also allows a belief agent to specify more than one measurements for an uncertainty; however, each of these measurements is enforced to take different kinds of measures (UR33 and its OCL constraint in Appendix B).

U-RUCM (along with its editor) also provides the capability of specifying measurement statements as belief special sentences in the sense that a measurement statement is divided into two parts: the measure and the value with the format of *measure::value*. For example, as shown in Figure 2, one measurement statement corresponding to the uncertainty is specified as *UModel.Measure.Probability::98%*. Notice that the measure taxonomy of *U-Model* (Appendix A.3) has been embedded as part of the U-RUCM editor. Therefore, when typing "UModel.Measure.", all the measures of the taxonomy will automatically be listed in a drop list for selection. Based on this format, a measurement statement can be automatically parsed, and an instance of *Measurement* will be

automatically created and a value will be assigned to the *value* attribute of the measurement statement.

5 TOOL SUPPORT AND METHODOLOGY

5.1 Tool Support

BUCSs are specified in the U-RUCM editor, which is implemented in a modeling framework, called the *Lightweight Modeling Framework* (LMF [28]). This framework implements functionality similar to those of the Eclipse Modeling Framework (EMF), but with a lightweight design with the aim of reducing tight coupling with Eclipse to facilitate easier porting to other platforms. LMF has two editors: a reflective model editor and a metamodel editor. The LMF Reflective Editor is a simple model editor implemented with the LMF metamodel reflection mechanism. The metamodel editor allows editing a LMF metamodel.

BUCSs specified with the editor can be automatically formalized into instances of *BeliefUCMeta* concepts. In the past, we have developed the transformation from RUCM to UCMeta, based on natural language processing techniques [8]. The transformation from U-RUCM to *BeliefUCMeta* is just an extension of the transformation from RUCM to UCMeta. The formalized specifications can be directly used for performing different kinds of analyses and generations of other artifacts when needed.

We have made a video to demonstrate the U-RUCM editor and the formalization from U-RUCM to *BeliefUCMeta*, along with the metamodel of *BeliefUCMeta*, UCMeta and *U-Model* in [15] for references. Note that Figure 1, Figure 2, Figure 3 and Figure 20 also demonstrate the user interfaces of the tool.

5.2 Methodology

Though U-RUCM can be used in many different ways, we recommend one methodology based on our own experience. The methodology is specified with UML activity diagrams, i.e., Figures 9 -12, integrating all restriction rules (Table 2 and Table 3). It starts with the creation of a use case model, specifying the actors, use cases, and relationships among them (Figure 9). The belief agents, in this case, are the requirements engineers who capture the information including indeterminacy sources, evidence, and uncertainty degrees from various stakeholders. In addition, requirements engineers are required to not only follow the rules associated to corresponding steps, but also make decisions (denoted as black diamonds) about what next step is. Note that, it is always possible to revisit the initial specifications when evidence or indeterminacy sources are required to be updated (i.e., added and removed). For instance, after D4, we leave a decision for users to make about whether they need to add more elements after specifying a BUCS. Similarly, once new elements are added, a user can also revisit the BUCS. When a model element is deleted, it results in deleting all the elements owned by the deleted element. In addition, our tool gives warnings attached on all relevant elements referring to the deleted ones, then a user can revisit the part of the BUCS where warnings are displayed. For instance,

once an indeterminacy source is removed, related *IndeterminacyKnowledge*s will refer to *none, i.e., null*. As a result, warnings are instantly tagged on all related fields where the indeterminacy source was referenced in a BUCS, BS and/or uncertainty.

Note that we did not specialize removal or modification actions in activity diagrams since their actions are similar as for creation. More specifically, all modification actions follow same actions as for creation, e.g., follow S4 (Figure 9) for modifying an existing BUCS. For removal actions, it can be treated as two steps, 1) one is a simple action to remove selected model elements, and 2) another is to modify elements led by the removed ones. Since our tool supports to gives warnings for this case, and thus user can also handle the removal actions using the presented methodology.

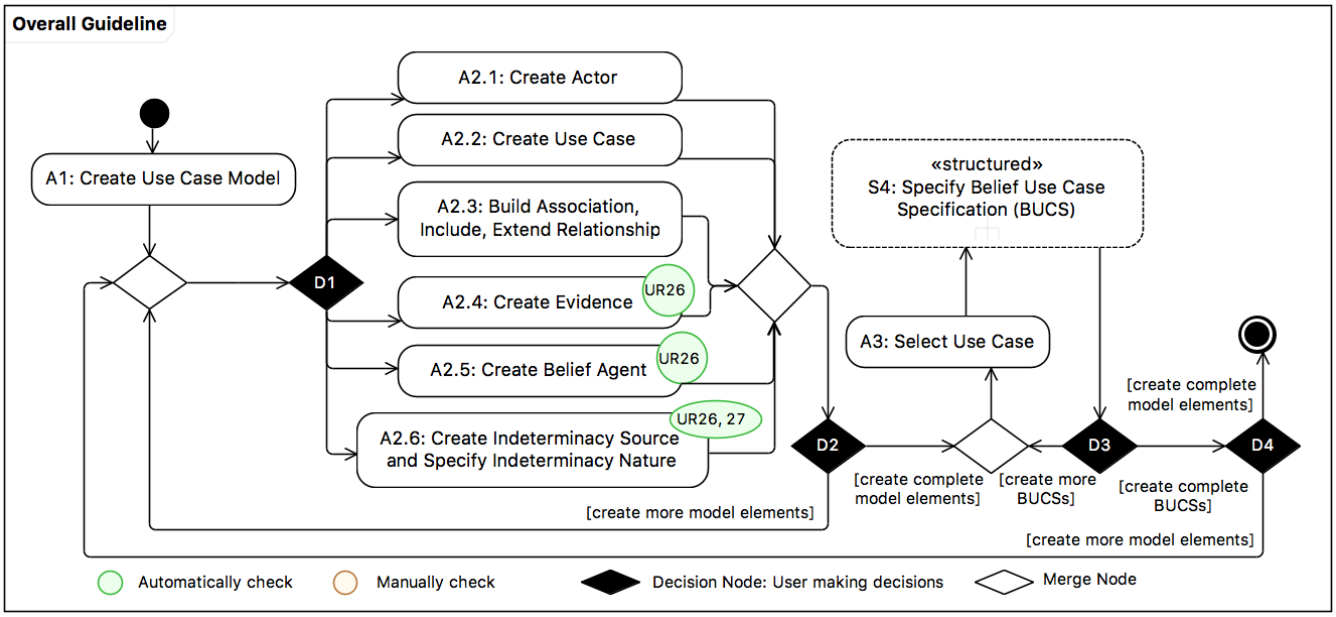


Figure 9. Methodology for creating a use case model (in UML Activity Diagram)

When the overall context of a belief use case model is established, one can start to develop a BUCS for each use case. The key steps of developing BUCSs are presented as a UML activity diagram in Figure 10. There is no particular order for specifying primary and secondary actors, belief agents. We recommend a sequence for guiding requirements engineers through the process that proceeds from simple tasks to more complicated ones. Specifying flows of events is the most challenging task, as it requires a lot of careful analysis, discussions, and design. The process is always iterative (i.e., another revisit), although we do not show this in Figure 10 for reasons of simplicity.

When specifying belief alternative flows (Figure 10), belief global alternative flows are often used to specify exceptions and behaviors crosscutting all the steps of a reference flow. The key task here is to identify the proper branching condition. If one needs to refer to one or more (but not all) steps of a reference flow, belief specific or bounded alternative flows can be created. As discussed in Section 4.4, U-RUCM provides four different ways of specifying belief alternative flows and related restriction

rules (e.g., alternative sentences only appear in URFS alternative flows) should be applied when using U-RUCM in this aspect. In our current implementation of the editor, we have partially enforced these restriction rules (all the rules that can be automatically checked by the editor have been specified with OCL constraints shown in Appendix B) such that chances of violating them are eliminated. By definition, URFS and RFS are different and therefore should be applied in different situations, as discussed in Section 4.4. We highly recommend using URFS to identify uncertain alternative flows only after the entire structure of flows of events (using RFS) of a BUCS is defined.

Each flow of events consists of a set of steps, which are specified as belief sentences (Figure 11). For each belief sentence, one should refer to one or more relevant indeterminacy sources and evidence, based on which, one can define the belief degree and associated uncertainties. The essential activity of specifying a belief sentence is about specifying associated uncertainties if there are any. The key steps of specifying uncertainties of the *NLUncertainty* type for belief sentences are presented as a UML activity diagram in Figure 12.

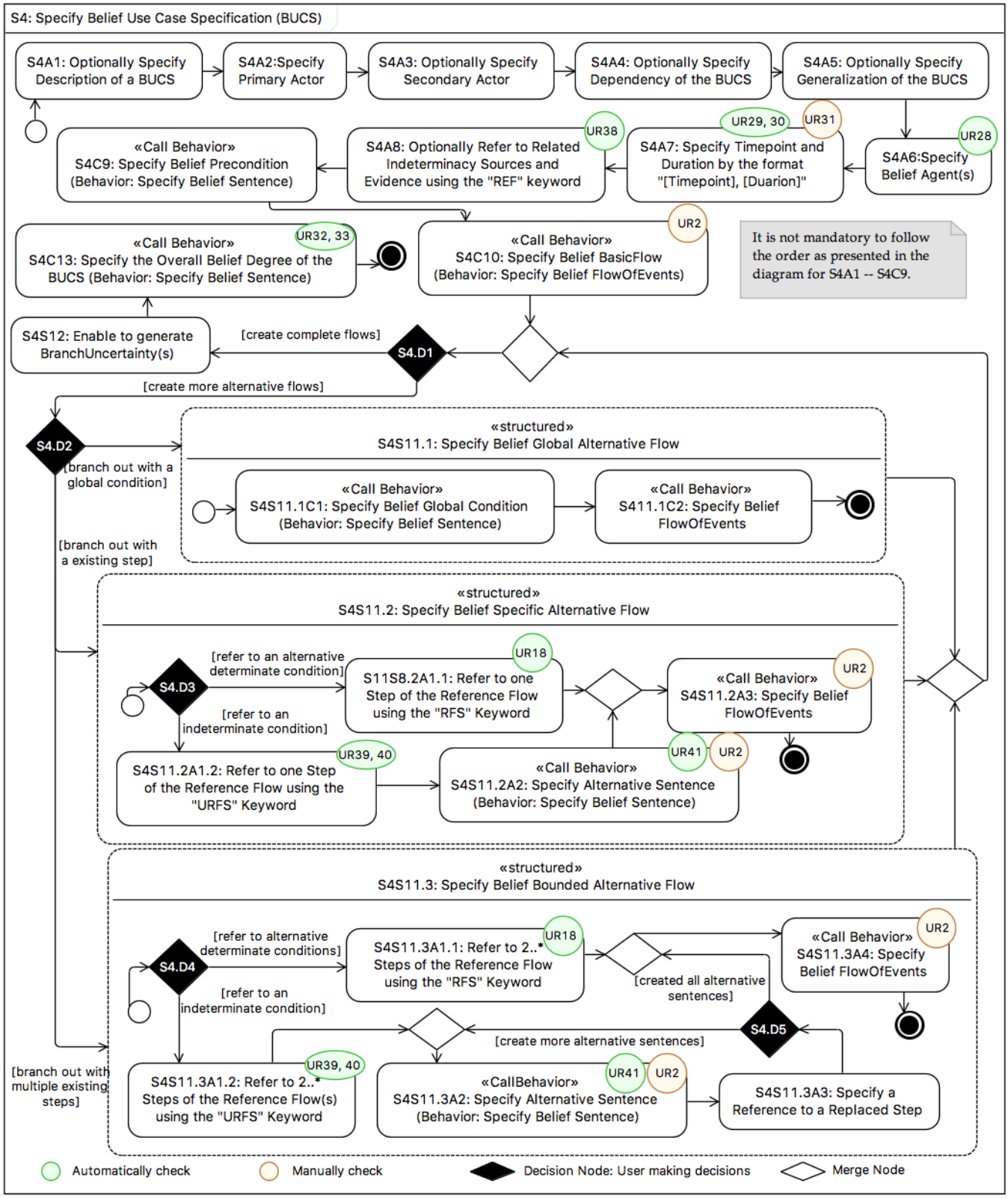


Figure 10. Methodology for specifying BUCSs (in UML Activity Diagram)

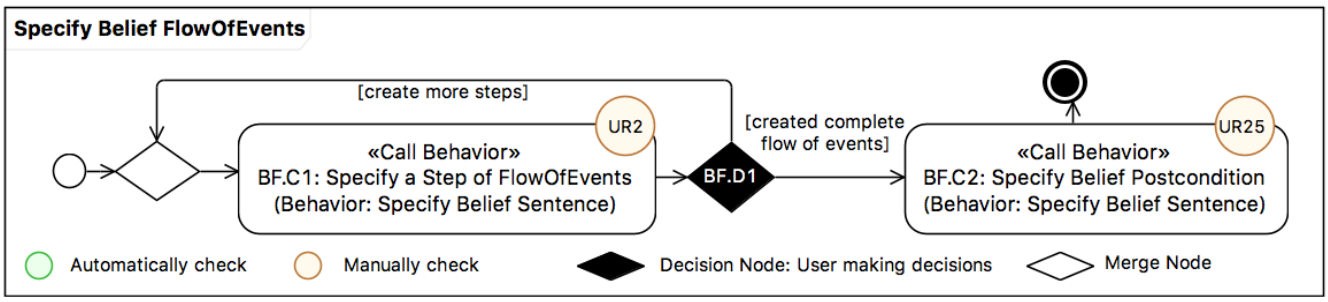


Figure 11. Methodology for specifying steps in a BeliefFlowOfEvents (in UML Activity Diagram)

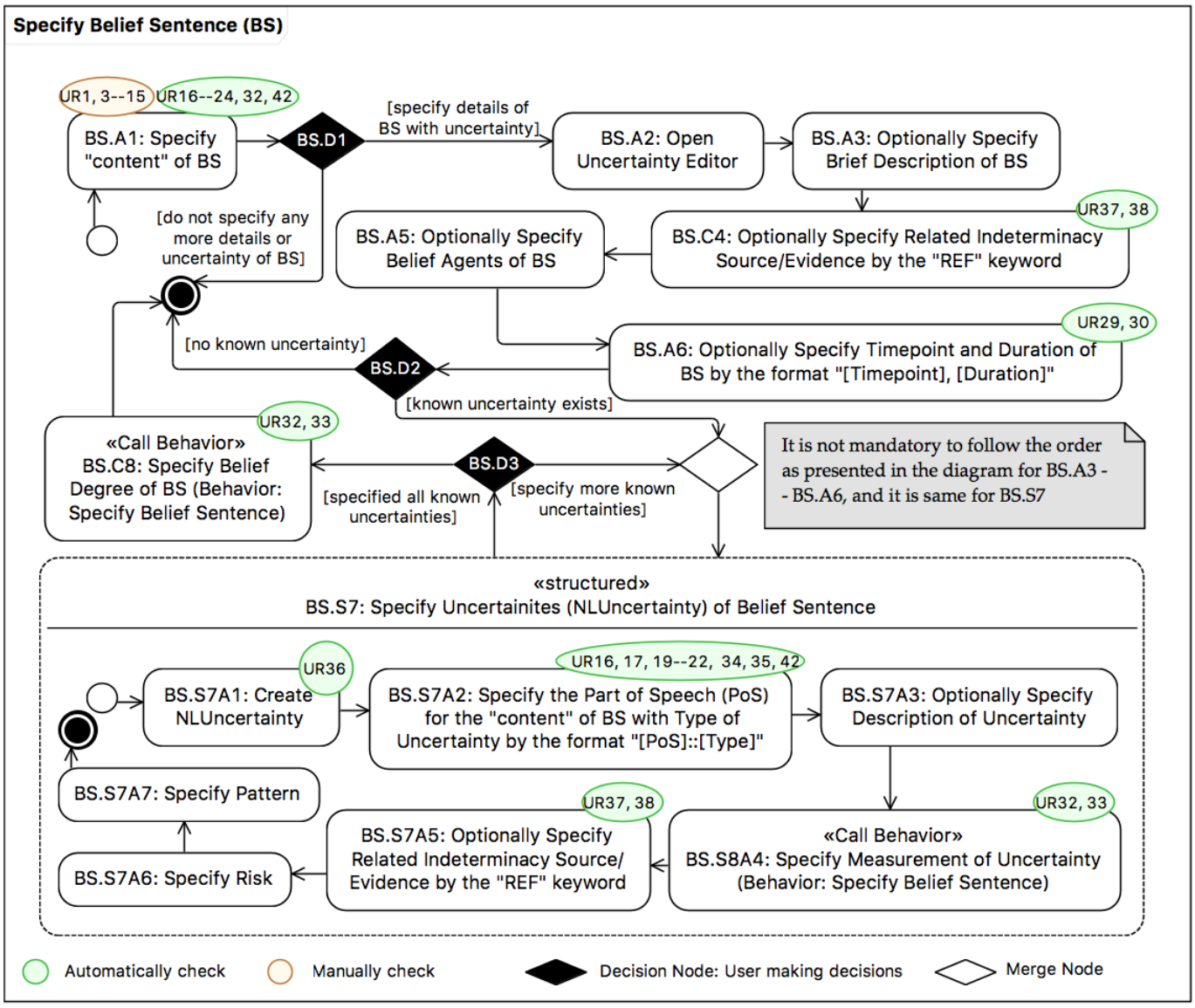


Figure 12. Methodology for specifying a BS (in UML Activity Diagram)

As discussed in Section 4.6, uncertainties can be characterized precisely with pattern and risk information and can be measured in different ways. In practice, it is not always possible to obtain and enter all of this information at once. So, a rule of thumb is to first identify as many uncertainties as possible and only then refine them with more detailed information. The recommendation is also based

on the fact that it is more important, at the requirements specification stage, to spend time (which is often limited) on identifying more uncertainties than elaborating on details of already identified uncertainties. If one wants to elaborate on the details of a previously identified uncertainty, it is recommended to start from identifying associated indeterminacy sources. This is because identifying indeterminacy sources (e.g., *REF Broken Control Panel (Button or Screen)* in Figure 2) might lead to the discovery of previously-unknown uncertainties that might be caused by this indeterminacy (e.g., uncertainties due to a broken control panel).

Since measurement statement is one type of belief sentence, specifying measurement statements is same as shown in Figure 12. Thus, if a measurement statement contains uncertainties, then a procedure same to the one for specifying *NLUncertainty* for belief sentences can be followed.

6 EVALUATION

The overall objective of the evaluation was to assess, in an industrial setting, whether U-RUCM was effective regarding facilitating the development of use case models with the explicit focus on uncertainty. In *U-Model* Section 6.1, we briefly describe the two industrial case studies. In Section 6.2, we present the context, design, and execution of the evaluation. Results of the case studies are presented in Section 6.3. In Section 6.4, we share our experience, lessons learned and identify future challenges.

6.1 Case Studies

One of the two industrial case studies involved the Automated Warehouse (AW) from ULMA Handling Systems. These complex systems serve to monitor, control, and manage warehouses for goods of different types, such as food and beverages and textiles. Each handling facility (e.g., crane, conveyor) forms a physical unit, and together they are dedicated to one handling system application (e.g., storage).

The second industrial case study used the Geo Sports (GS) system from Future Position X [29], Sweden. This system measures the performance of an individual or a team as well as the conditions of athletes over a sustained period in actual game environments (e.g., a soccer field). The measurements (e.g., heartbeat, speed, location) are made continuously and in real time using geo-position sensors during both training and competition. These measurements are communicated during a game via a receiver station to the OpenField[5] system, where coaches can monitor them at runtime and take actions when needed. In addition, these measurements can also be used offline for analyses, for example, aimed at improving the performance of an individual player or a team. Our case study involved

[5]A cloud-based analytics platform for reporting and presenting data (see https://www.catapultsports.com/products/openfield for more information).

Bandy, a form of ice hockey played predominantly in Northern Europe and Russia. To the best of our knowledge, this project was the first to monitor sports on ice using sensors. We consider GS as a human-in-the-loop CPS [30].

6.2 Context, Design, and Execution of Evaluation

The work was conducted in the context of the U-Test[6] project. The development of U-RUCM is an iterative process and interwoven with the activities of developing *U-Model*, eliciting, refining and validating uncertainty requirements of the two industrial case studies. Both researchers and industrial partners were involved in the process, during which intermediate versions of U-RUCM were developed. We consider that being transparent and therefore reporting the process in the paper are important since it provides an opportunity for readers to comprehend the rigorousness of the process and therefore gain confidence on the derived U-RUCM methodology. Moreover, interested readers might consider following similar procedures to develop similar approaches in similar contexts.

The development and validation procedure of U-RUCM is shown in Figure 13 comprising of two parallel processes: 1) related to the development of U-RUCM mainly conducted by the researchers; 2) validation of uncertainty requirements mainly performed by the industrial partners. At the start of the project (before developing U-RUCM), RUCM was introduced to both industrial partners (i.e., ULMA and FPX) as a means for eliciting and specifying the initial versions of their uncertainty requirements as shown as step *B1* in Figure 13. In Table 13 of Appendix C, we provide a sanitized example of UCSs capturing uncertainty requirements with an extended RUCM template. This example was documented by our industrial partners. Results of this activity are 20 use cases for each case study, 93 RUCM flows of events (52 for AW and 41 for GS), as shown in column *RUCM Model* of Table 6. In total, the RUCM model for AW had 229 sentences, while the GS model had 256. About uncertainties specified in the RUCM models, 33 (for AW) and 26 (for GS) sets of steps of flows of events describing alternative scenarios were considered as involving uncertainties. It is important to point it out that at this stage, uncertainty requirements (i.e., *Uncertainty Req.V1*, Figure 13) were specified in a coarse-grained manner, which clearly justified the need of developing U-RUCM.

Second, we conducted a questionnaire-based survey to collect data to detail and quantify the identified uncertainties (step *A1*, Figure 13). During this non-trivial process, RUCM was deemed adequate to provide initial data, but it captured uncertainty requirements at a coarse-grained level. The output of this step is the initial version of *U-RUCM V1*, Figure 13. The questionnaire was derived from the RUCM models developed by the industrial partners (*Uncertainty Req.V1*, Figure 13) and it was designed for each use case specification. As summarized in Table 5, the first two types of

[6] http://www.u-test.eu/

questions were meant for inspecting a UCS from the use case modeling perspective and they are generic; the third to sixth types of questions were proposed with the aim to elicit new uncertainty requirements; the last five types of questions were proposed with the aim to detail already specified uncertainty requirements. To get a concrete idea, we provide in Appendix C a list of questions derived for a particular use case (the original version of which is given in Table 13, a slightly improved version (refined by researchers with the RUCM editor) of which are provided in Figure 18 and Figure 19) as an example.

According to the questionnaire and comments provided by researchers (step *A1*), industrial partners developed the second version of the uncertainty requirements (*Uncertainty Req. V2*), which are specified using *U-RUCM V1*. Subsequently, two onsite workshops, i.e., one for each partner were conducted to further refine the collected requirements, i.e., *Uncertainty Req V2* (step *A2/B3*). The output of the step is *Uncertainty Req. V3*, which is input to the *A3* step for refining *U-RUCM V1* into *U-RUCM V2* and developing and formalizing *Uncertainty Req. V4* (step *A3*). *U-RUCM V2* is the final version presented in this paper, and the current U-RUCM editor was developed based on this version of the U-RUCM methodology, and *Uncertainty Req. V4* is the final version of uncertainty requirements, which is specified with *U-RUCM V2*. The collected results of the evaluation are based on the comparison of *Uncertainty Req. V1* and *Uncertainty Req. V4* as presented in Section 6.3. We also provide a sanitized example of the AW industrial case study in Appendix D for reference, which was a final use case specification specified with the latest version of the U-RUCM tool.

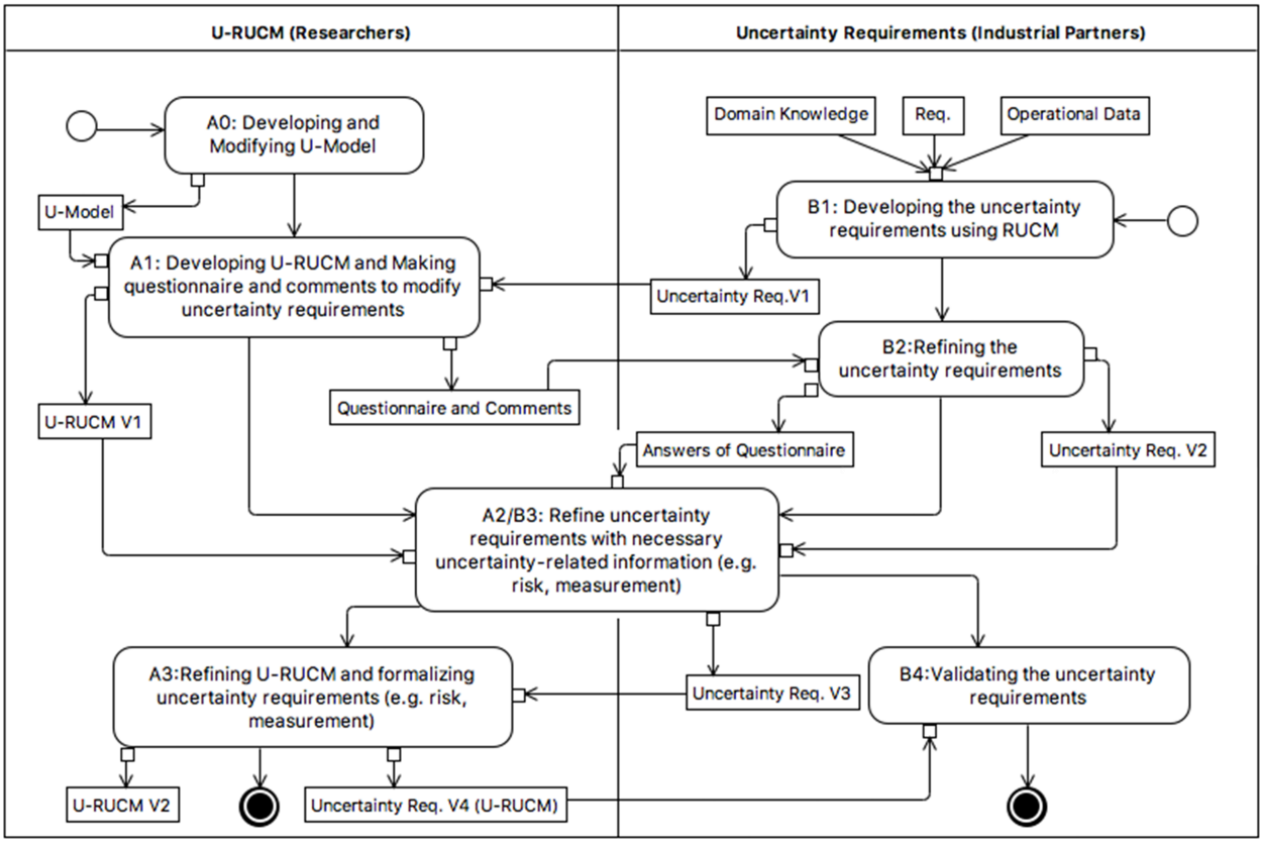


Figure 13. Development and Validation of U-RUCM

Table 5. Design of the questionnaire (A1, Figure 13)

#	Explanation
1	Inquiry the boundary of the system to define actors in the use case model.
2	Check the completeness of the flow of events of each use case specification.
3	Inquiry the existence of sources related to an actor.
4	Inquiry the existence of potential uncertainties related to system properties or behaviors.
5	Inquiry existence of the potential uncertainties regarding time, nature and human being.
6	Inquiry if a potential uncertainty is valid by checking if it is derived based on system properties or behaviors.
7	Inquiry the completeness of the types of uncertainties defined in U-Model.
8	Inquiry the type of a specified uncertainty.
9	Inquiry the measure and measurement of a specified uncertainty.
10	Inquiry the risk of a specified uncertainty.
11	Inquiry the evidence to support the specified measurement and risk of a specified uncertainty.

6.3 Results

As previously discussed, all the RUCM models developed by the industrial partners were refined using U-RUCM to capture all the identified uncertainties. Descriptive statistics of the resulted U-RUCM models, i.e., *Uncertainty Req. V1*, are reported in the *Key RUCM Element* columns of Table 6. The table shows how many elements were added, modified, and removed during the process for the two case studies reported in the *Refinement* column. The elements existing in the final U-RUCM models, i.e., *Uncertainty Req. V4*, are reported in the U-*RUCM Model* column.

Recall that U-RUCM realizes the *Uncertainty* concept of *U-Model* by three concrete means: 1) *NLUncertainty* for belief sentences (Section 4.6.1), 2) *BranchUncertainty* for possible executions of BUCSs from the beginning to end (Section 4.6.2), 3) uncertainties in flows of events captured via URFS and alternative flows (Section 4.4). We applied these four U-RUCM mechanisms systematically by following the guidelines described in Section 5 and then carefully examined all the specified BUCSs to refine the U-RUCM models further.

As shown in Table 6, we refactored the design of the RUCM use case model of AW by merging three use cases describing similar scenarios into one, which led to the deletion of 2 use cases (as shown in the table). We also added two use cases to the RUCM model of GS as the result of the refactoring, as these two use cases can be invoked (via the include relationships) by multiple use cases.

Table 6 also indicates that three uncertainties in the AW RUCM model were removed and four from the GS model. This was because: 1) We optimized the design of the use case model by removing duplicated uncertainties, i.e., one from AW and two from GS. For example, uncertainties describing improper wearing of positioning devices is the same for both indoor and outdoor games; 2) We identified uncertainties from the RUCM models that are indeterminacy sources, which were two for AW and two for GS. For example, the long distance between a positioning device and the satellites is an indeterminacy source (previously identified as an uncertainty), which can lead to the failure of locating the satellites with insufficient resolution nature.

The uncertainty-specific concepts *Indeterminacy Source*, *Alternative Sentence,* and *BranchUncertainty* were only introduced in U-RUCM. Consequently, there were no corresponding elements in the RUCM models. After carefully going through details of the RUCM models using steps described previously, we derived a total of 23 indeterminacy sources for AW and 18 for GS, 45 alternative sentences for AW and 76 for GS. Furthermore, we discovered 32 instances of *NLUncertainty* for AW and 48 for GS. These turned out to be cases of "unknown knowns" for our industrial partners, that is, tacit knowledge that was not explicit initially. This activity led to the addition of 18 belief flows of events for AW and 31 for GS, 72 new branch uncertainties for AW and 89 for GS, and 43 additional alternative flows with URFS applied for AW and 48 for GS.

Table 6. Descriptive Statistics of the RUCM Models, U-RUCM Models and Refinements

AW						Key U-RUCM Elements	GS					
Key RUCM Elements		Refinement			U-RUCM Model		U-RUCM Model	Refinement			Key RUCM Elements	
		#A	#M	#R				#A	#M	#R		
Use Case	20	0	20	2	**18**	Use Case	**23**	3	20	0	20	Use Case
FlowOfEvents	52	18	23	4	**66**	(Belief) Flow Of Events	**71**	31	21	1	41	FlowOfEvents
Sentence	229	63	56	16	**276**	(Belief) Sentence	**348**	97	46	5	256	Sentence
		45	0	0	**45**	Alternative Sentence	**76**	76	0	0		
Uncertainty	33	32	18	3(1,2)+	**62**	(NL)Uncertainty	**70**	48	15	4 (2,2)+	26	Uncertainty
		72	0	0	**72**	BranchUncertainty	**89**	89	0	0		
		43		0	**43**	URFS	**48**	48	0	0		
		23	0	0	**23**	Indeterminacy Source	**18**	18	0	0		

#A is the number of added elements, #M is the number of modified elements, and #R is the number of removed elements;
*(n,m)+ -- n is the number of uncertainties removed due to refactoring; m is the number of uncertainties that are changed to

In summary, the total numbers of the instances of metaclasses *NLUncertainty* and *BranchUncertainty* of *U-Model*, populated for each of the industrial case studies are 62+72=134 for AW and 70+89=159 for GS. When comparing this with their corresponding "rough" RUCM models, we conclude that, by using U-RUCM, we were able to significantly enhance the extent and precision of modeling uncertainties in requirements (i.e., (134-33)/33=306% for AW and (159-26)/26=512% for GS). This suggests that U-RUCM is an important improvement in dealing with uncertainty in requirements engineering. More specifically, to compare with RUCM, U-RUCM provides a way to elicit, specify and model 1) uncertainty alternative flows describing scenarios that the belief agent lacks confidence about which flow of events occurs given an indeterminacy source, 2) uncertainty alternative actions where the belief agent lacks confidence about which action occurs, given an indeterminacy source, and 3) measurements of uncertainty, such as probability, interval, which are useful when analyzing/reasoning uncertainty.

In our EU project, the U-RUCM models capturing uncertainty requirements were used as the input for developing the test ready models [16, 18] represented as UML class diagrams and state machines using *UncerTum* (see Section 2.1). The test ready models were used to generate test cases, which were then executed successfully in actual systems [19]. There are clear correspondences between the scenarios and uncertainties defined in the test ready models and the ones defined in the U-RUCM models. Note that uncertainty measurements, risk information, among others, were directly migrated from the U-RUCM models to the test ready models. Doing so significantly reduced the effort required to develop the test ready models. Also, throughout the project, the elicited and validated uncertainty requirements were used as the communication medium among the partners.

6.4 Experience, Lessons Learned, and Future Challenges

This section presents our experience, lessons learned, and future challenges.

Identifying common uncertainties, measurements, and indeterminacy sources. From the GS case study,

we noted that human behavior was the key indeterminacy source of uncertainties, due to incorrect interactions with the system. For the AW case study, on the other hand, uncertainties and indeterminacy sources centered mainly on the data communications between control units and their controlled devices. From these types of observations, we can conclude that it is possible in principle to identify common sources, types, and measurements of uncertainties that occur in a given domain or even across domains. This knowledge can be then used to define *reusable* uncertainty specifications and their corresponding behaviors.

Specializing U-RUCM. RUCM can be specialized for different purposes and domains. For example, in another research project, we developed a version of RUCM specifically for real-time systems [31]. In such cases, the standard RUCM template and keywords were extended to allow the specification of time constraints. These are also subject to uncertainty. Based on that, we anticipate that U-RUCM will also need to be extended to specific domains.

Learning about uncertainty by applying U-RUCM. In the past, we experienced that one can learn how to better design use case models by using RUCM. This is why RUCM is used as a teaching method for requirements engineering and software engineering courses both at the undergraduate and graduate levels[7]. Similarly, based on the results of this project, we surmise that it is possible to gain more precise and more direct understanding of both uncertainty and indeterminacy sources by using U-RUCM.

Automated, scalable, and systematic reasoning. For effective coping with uncertainty, automated/semi-automated reasoning about uncertainty and indeterminacy sources can indeed be helpful. This is because, for any non-trivial system, a use case model might be large and may contain a large number of potentially inter-related uncertainties. From our experience during the initial phases of our study when we were not using U-RUCM, we learned that unassisted human reasoning tends to be time-consuming and unsystematic. This is why we chose a more formal approach when developing U-RUCM – via the *BeliefUCMeta* metamodel – which provides a formal foundation for future, automated reasoning techniques.

Harvesting the benefits of natural language processing techniques. When deriving U-RUCM and performing the two industrial case studies, we noticed that there is an opportunity to further refine *NLUncertainty*, the core concept for representing uncertainties in belief sentences (see Section 4.6). The general idea here is to rely on natural language processing techniques to automatically identify grammatical structures (e.g., Subject), PoSs (e.g., Verb), sentence structures (e.g., Subject-Verb-Object), and/or sentence semantics (e.g., an actor) sends a request to the system in belief sentences. With this,

[7] http://www.zen-tools.com/rucm/index.html

heuristics can be defined to automatically identify potential uncertainties and/or verify already specified ones in belief sentences. For example, a verb of the predicator of a sentence might have an *Occurrence* uncertainty associated with it. A noun being the direct object of a simple sentence might be associated with a *Content* uncertainty.

Reckoning on branch uncertainties. It may be possible to automatically derive values of branch uncertainties (A branch uncertainty indicates a belief agent's confidence in the possibility that the execution of the use case takes this particular branch.) At the very least, branch uncertainties can help to 1) identify critical paths to reduce uncertainties or perform risk analyses (if the postcondition that a branch leads to is considered as the consequence of the branch), 2) verify the overall belief degree that a belief agent has in a belief specification, and 3) derive test cases targeting branches particularly with high uncertainty. This is a possible avenue of further research.

Systematically discovering unknown known indeterminacy sources and uncertainties and transforming them into known unknown uncertainties and known known indeterminacy sources. As the case study results showed, it is possible to systematically identify previously unknown known based on already-specified (known) uncertainties and indeterminacy sources. A systematic methodology (ideally with tool support) can be followed to identify more unknown knowns and currently known unknown uncertainties (e.g., by combining already identified uncertainties that are associated with the same part of system behavior).

Transforming U-RUCM models into other downstream artifacts. To maximize the benefit of U-RUCM models, one possibility is to transform them automatically or semi-automatically into other artifacts that need to be developed during system development. For example, U-RUCM models can be transformed into UML state machines via the UUP profile (Section 2.1), for supporting MBT of CPSs under uncertainties. This is feasible as RUCM models can be transformed into UML models and test cases (Section 2.3).

7 RELATED WORK

Runtime detection, monitoring, reasoning, and managing of requirements, generally referred to as being *requirements-aware*, is necessary for self-adaptive systems [32], due to inherent changes in operational environments and contextual uncertainties. For this purpose, RELAX [2, 3] – a representative requirements specification and reasoning solution – was proposed to support the development of requirements for dynamically adaptive systems with environmental uncertainty. RELAX consists of a set of keywords, which are classified into *modal*, *temporal*, *ordinal* and *uncertainty* operators. Uncertainty factors aim to indicate where a relaxation of requirements is warranted and, therefore, adaptive behavior is needed. Based on a structured natural language based notation, the

authors also proposed a methodology for relaxing requirement statements with the RELAX keywords. Also, RELAX requirements can be formalized using fuzzy logic and reasoning can be performed, when needed.

RELAX has been also integrated with goal-modeling notations (i.e., KAOS [33]) to allow for fuzzy goals [2]. Along the same line, Luciano et al. [4] proposed FLAGS for enabling the specification of adaptive goals, which are of two types: *crisp* goals (specified via linear temporal logic) and *fuzzy* goals (specified using a fuzzy temporal logic). Chen et al. [34] proposed a goal-driven self-optimization framework to handle three different types of uncertainty in goal models: *contribution*, *preference,* and *effect* uncertainty. ReAssuRE was proposed by Welsh et al. [5] to attach *claims* to softgoal contribution links of goal models, with the aim to record the rationale for selecting a goal realization strategy when the optimum choice is uncertain. Later on, Ramirez et al. [35] integrated ReAssuRE with RELAX to assess the validity of claims at runtime, for dealing with environmental uncertainty in dynamic adaptive systems.

Compared to these goal-based approaches, U-RUCM is more generic, as it is not targeting dynamic adaptive systems in particular. Second, U-RUCM is built on a use case modeling methodology, such that it can naturally facilitate the specification of uncertain alternative scenarios. Furthermore, U-RUCM also enables the specification of various types of uncertainties (e.g., *Time*, *Occurrence*), more precise characterization of uncertainties with information such as *Pattern*, and the ability to quantify uncertainties in different ways (e.g., *Probability*, *Fuzziness*). Currently, U-RUCM has a dedicated template for specifying uncertainty. In the future, it would be useful to investigate using keywords (similar to RELAX) to reduce the effort in specifying uncertainties.

Uncertainty is also considered as an important factor that complicates early requirements definition and decision making. Salay et al. [6] proposed the *MAVO* annotations for modeling uncertainty in requirements engineering models, based on the concept of *partial models* (with their properties checked as *True*, *False* or *Maybe*) [36]. The *MAVO partiality* annotations consist of: *May partiality* (indicating that an element should exist in the model), *Abs partiality* (indicating that an element is a collection of elements), and *Var partiality* (indicating that it is unclear if an element should be merged with others). Famelis and Santosa [7] proposed to use colored Entity-Relation models for explicitly capturing the MAVO partiality, as well as *Points of Uncertainty*, a concept representing a specific decision about which there is uncertainty. Compared to these partial-model solutions, U-RUCM is systematically derived from *U-Model*, and it is integrated with RUCM, which enables the specification of uncertain alternative scenarios.

Uncertainty can hinder organizations in making strategic decisions due to, for example, uncertain stakeholders' goals and priorities. In this context, uncertainty is defined as the lack of knowledge of

the consequences of decision alternatives. Letier et al. [37] proposed ways for reasoning about uncertainties to support early requirements and architecture decision analysis. Uncertainties are represented as probability distributions, while Monte-Carlo simulations are used for simulating the impact of alternative decisions. That paper, however, does not provide a solution for uncertainty specification and elicitation. Similarly, Esfahani et al. [38] proposed GuideArch, a framework for the quantitative exploration of the architectural solution space under uncertainty, which is based on fuzzy mathematical methods for reasoning about uncertainty. Although these works support means for reasoning, simulation, and exploration in the presence of uncertainty, they lack the capability of specifying and modeling of uncertainty in the context of requirements engineering.

8 Conclusion and Future Work

The impact of uncertainty, which is increasingly being recognized as an inherent and crucial property of non-trivial software-intensive systems (e.g., CPSs), needs to be better understood and addressed explicitly in all phases of system development. In particular, it has to be explored and characterized as much as possible during requirements engineering (e.g., elicitation, specification, and verification). Use case modeling is a well-known and commonly applied requirements specification/modeling method in practice. Specifying uncertainty as part of use case models is therefore particularly useful. In this paper, we described a methodology and a corresponding tool (U-RUCM) for helping practitioners to specify uncertainties in requirements as part of use case models.

U-RUCM originated in the context of the EU project [39], which involved a consortium of nine partners. The initial version of the uncertainty requirements was developed by our industrial partners using the basic RUCM methodology, on which U-RUCM was founded. After refining the RUCM models, by applying the U-RUCM methodology, we successfully identified and specified more than 300% and 500% (previously unknown) uncertainty requirements for the two case studies. The resulting U-RUCM models were used as a reference to develop test ready models for generating executable test cases to test the two industrial applications. As users of U-RUCM ourselves during the evaluation (i.e., case studies), we gained invaluable experience about its use and future potential.

In the future, we plan to enrich the capabilities of U-RUCM from the following aspects: 1) identifying and specifying common uncertainties within and across domains, 2) specializing U-RUCM for the real-time domain, 3) reasoning uncertainties at various levels such as at the sentence level by relying on NL techniques, the structure level of use case specifications by, e.g., analyzing the cause-effect of the sequential order of steps of flows of events, and 4) automated transformation of U-RUCM models into other artifacts such as test cases. We also plan to conduct controlled experiments to evaluate the applicability of U-RUCM and conduct more industrial case studies to understand its

potential and limitations better.

ACKNOWLEDGMENT

This research was supported by the EU Horizon 2020 funded project (Testing Cyber-Physical Systems under Uncertainty, Project Number: 645463). Tao Yue and Shaukat Ali are also supported by RCN funded Zen-Configurator project, RFF Hovedstaden funded MBE-CR project, RCN funded MBT4CPS project, RCN funded Certus SFI and the EU COST action MPM4CPS. Man Zhang is funded by the EU Horizon 2020 funded project (Testing Cyber-Physical Systems under Uncertainty, Project Number: 645463).

REFERENCES

[1] K. Bittner, and I. Spence, Use Case Modeling, Addison-Wesley, 2003.

[2] B. H. C. Cheng, P. Sawyer, N. Bencomo, and J. Whittle, A Goal-Based Modeling Approach to Develop Requirements of an Adaptive System with Environmental Uncertainty, *Model Driven Engineering Languages and Systems: 12th International Conference, MODELS 2009, Denver, CO, USA, October 4-9, 2009. Proceedings*, A. Schürr and B. Selic, eds., pp. 468-483, Berlin, Heidelberg: Springer Berlin Heidelberg, 2009.

[3] J. Whittle, P. Sawyer, N. Bencomo, B. H. C. Cheng, and J.-M. Bruel, RELAX: a language to address uncertainty in self-adaptive systems requirement, Requirements Engineering, vol. 15, no. 2 (2010) 177-196.

[4] L. Baresi, L. Pasquale, and P. Spoletini, Fuzzy goals for requirements-driven adaptation, in: 2010 18th IEEE International Requirements Engineering Conference. pp. 125-134, 2010.

[5] K. Welsh, P. Sawyer, and N. Bencomo, Towards requirements aware systems: Run-time resolution of design-time assumptions, in: Automated Software Engineering (ASE), 2011 26th IEEE/ACM International Conference on. pp. 560-563, 2011.

[6] R. Salay, M. Chechik, J. Horkoff, and A. D. Sandro, Managing requirements uncertainty with partial models, Requirements Engineering, vol. 18, no. 2 (2013) 107-128, 10.1007/s00766-013-0170-y.

[7] M. Famelis, and S. Santosa, MAV-Vis: a notation for model uncertainty, in: Modeling in Software Engineering (MiSE), 2013 5th International Workshop on. pp. 7-12, 2013.

[8] T. Yue, L. C. Briand, and Y. Labiche, aToucan: An Automated Framework to Derive UML Analysis Models from Use Case Models, ACM Transactions on Software Engineering and Methodology (TOSEM), vol. 24, no. 3 (2015) 13.

[9] T. Yue, L. C. Briand, and Y. Labiche, Facilitating the transition from use case models to analysis models: Approach and experiments, ACM Transactions on Software Engineering and Methodology (TOSEM), vol. 22, no. 1 (2013) 5.

[10] T. Yue, S. Ali, and M. Zhang, "Applying A Restricted Natural Language Based Test Case Generation Approach in An Industrial Context," *International Symposium on Software Testing and Analysis (ISSTA)*, 2015.

[11] H. Zhang, T. Yue, S. Ali, and C. Liu, Facilitating requirements inspection with search-based selection of diverse use case scenarios, in: proceedings of the 9th EAI International Conference on Bio-inspired Information and Communications Technologies (formerly BIONETICS) on 9th EAI International Conference on Bio-inspired Information and Communications Technologies (formerly BIONETICS). pp. 229-236, 2016.

[12] M. Zhang, T. Yue, S. Ali, H. Zhang, and J. Wu, A Systematic Approach to Automatically Derive Test Cases From Use Cases Specified in Restricted Natural Languages, in: D. Amyot, P. F. i. Casas and G. Mussbacher, eds. 8th System Analysis and Modelling Conference (SAM 2014), Switzerland, 2014.

[13] T. Yue, and S. Ali, Bridging the gap between requirements and aspect state machines to support non-functional testing: industrial case studies, in: European Conference on Modelling Foundations and Applications. pp. 133-145, 2012.

[14] C. Wang, F. Pastore, A. Goknil, L. Briand, and Z. Iqbal, Automatic generation of system test cases from use case specifications, in Proceedings of the 2015 International Symposium on Software Testing and Analysis, Baltimore, MD, USA, 2015, pp. 385-396.

[15] M. Zhang, B. Selic, S. Ali, T. Yue, O. Okariz, and R. Norgren, Understanding Uncertainty in Cyber-Physical Systems: A Conceptual Model, in: Proceedings of the 12th European Conference on Modelling Foundations and Applications (ECMFA). pp. 247-264, 2016.

[16] M. Zhang, S. Ali, T. Yue, and R. Norgren, An Integrated Modeling Framework to Facilitate Model-Based Testing of Cyber-Physical Systems under Uncertainty, 2016; https://www.simula.no/publications/integrated-modeling-framework-facilitate-model-based-testing-cyber-physical-systems.

[17] M. Zhang, S. Ali, T. Yue, R. Norgren, and O. Okariz, Uncertainty-Wise Cyber-Physical System test modeling, Software & Systems Modeling (2017), 2017/07/25, 10.1007/s10270-017-0609-6.

[18] M. Zhang, S. Ali, T. Yue, and R. Norgren, Interactively Evolving Test Ready Models with Uncertainty Developed for Testing Cyber-Physical Systems, Technical Report 2016-12, Simula Research Laboratory, 2016; https://www.simula.no/publications/interactively-evolving-test-ready-models-uncertainty-developed-testing-cyber-physical.

[19] M. Zhang, S. Ali, and T. Yue, Uncertainty-wise Test Case Generation and Minimization for Cyber-Physical Systems: A Multi-Objective Search-based Approach, Technical report 2016-13, Simula Research Laboratory, 2016; https://www.simula.no/publications/uncertainty-based-test-case-generation-and-minimization-cyber-physical-systems-multi.

[20] M. Zhang, S. Ali, T. Yue, and R. Norgre, Uncertainty-wise evolution of test ready models, Information and Software Technology (2017), http://dx.doi.org/10.1016/j.infsof.2017.03.003.

[21] R. S. Pressman, Software engineering: a practitioner's approach 7th edition, Palgrave Macmillan, 2010.

[22] J. Guo, J. White, G. Wang, J. Li, and Y. Wang, A genetic algorithm for optimized feature selection with resource constraints in software product lines, Journal of Systems and Software, vol. 84, no. 12 (2011) 2208-2221.

[23] OMG, "Meta Object Facility (MOF) Core Specification (Version 2.4.2)," 2014, http://www.omg.org/spec/MOF/2.4.2.

[24] T. Yue, L. Briand, and Y. Labiche, A Use Case Modeling Approach to Facilitate the Transition Towards Analysis Models: Concepts and Empirical Evaluation, in: A. Schürr and B. Selic, eds. Model Driven Engineering Languages and Systems (MODELS 2009), 2009.

[25] J. Wu, S. Ali, T. Yue, J. Tian, and C. Liu, Assessing the Quality of Industrial Avionics Software: An Extensive Empirical Evaluation, Empirical Software Engineering (2016).

[26] T. Yue, H. Zhang, S. Ali, and C. Liu, A Practical Use Case Modeling Approach to Specify Crosscutting Concerns: Industrial Applications, 2015.

[27] "U-RUCM: Specifying Uncertainty in Use Case Models," accessed; http://zen-tools.com/rucm/U_RUCM.html.

[28] G. Zhang, T. Yue, J. Wu, and S. Ali, Zen-RUCM: A Tool for Supporting a Comprehensive and Extensible Use Case Modeling Framework, in: Demos/Posters/StudentResearch@ MoDELS. pp. 41-45, 2013.

[29] "Future Position X," accessed 2017; http://www.fpx.se/.

[30] D. S. Nunes, P. Zhang, and J. S. Silva, A survey on human-in-the-loop applications towards an internet of all, IEEE Communications Surveys & Tutorials, vol. 17, no. 2 (2015) 944-965.

[31] H. Zhang, T. Yue, S. Ali, J. Wu, and C. Liu, A Restricted Natural Language Based Use Case Modeling Methodology for Real-Time Systems, in: 9th Workshop on Modelling in Software Engineering (MiSE'2017), 2017.

[32] P. Sawyer, N. Bencomo, J. Whittle, E. Letier, and A. Finkelstein, Requirements-Aware Systems: A Research Agenda for RE for Self-adaptive Systems, in: 2010 18th IEEE International Requirements Engineering Conference. pp. 95-103, 2010.

[33] A. Van Lamsweerde, Requirements engineering: from system goals to UML models to software specifications, Wiley Publishing, 2009.

[34] B. Chen, X. Peng, Y. Yu, and W. Zhao, Uncertainty handling in goal-driven self-optimization–limiting the negative effect on adaptation, Journal of Systems and Software, vol. 90 (2014) 114-127.

[35] A. J. Ramirez, B. H. C. Cheng, N. Bencomo, and P. Sawyer, Relaxing Claims: Coping with Uncertainty While Evaluating Assumptions at Run Time, *Model Driven Engineering Languages and Systems: 15th International Conference, MODELS 2012, Innsbruck, Austria, September 30–October 5, 2012. Proceedings*, R. B. France, J. Kazmeier, R. Breu and C. Atkinson, eds., pp. 53-69, Berlin, Heidelberg: Springer Berlin Heidelberg, 2012.

[36] M. Famelis, R. Salay, and M. Chechik, Partial models: Towards modeling and reasoning with uncertainty, in: Software Engineering (ICSE), 2012 34th International Conference on. pp. 573-583, 2012.

[37] E. Letier, D. Stefan, and E. T. Barr, Uncertainty, risk, and information value in software requirements and architecture, in: Proceedings of the 36th International Conference on Software Engineering. pp. 883-894, 2014.

[38] N. Esfahani, S. Malek, and K. Razavi, GuideArch: guiding the exploration of architectural solution space under uncertainty, in: 2013 35th International Conference on Software Engineering (ICSE). pp. 43-52, 2013.

[39] S. Ali, and T. Yue, U-Test: Evolving, Modelling and Testing Realistic Uncertain Behaviours of Cyber-Physical Systems, in: Proceedings of the IEEE 8th International Conference on Software Testing, Verification and Validation (ICST). pp. 1-2, 2015.

[40] P. R. Garvey, and Z. F. Lansdowne, Risk matrix: an approach for identifying, assessing, and ranking program risks, Air Force Journal of Logistics, vol. 22, no. 1 (1998) 18-21.

[41] G. Klir, Facets of systems science, Springer Science & Business Media, 2013.

Appendix A Definitions of the U-Model Concepts

To understand uncertainty, in our previous work [15], we defined the conceptual model, *U-Model*, to specify, classify and identify uncertainty and its associated concepts. U-RUCM presented in this paper is an implementation of *U-Model* to enable the specification of uncertainty in use case models. *U-Model* was published in [15] and we have added definitions of its concepts in this appendix to make the paper self-contained, which are organized into three parts: belief model, uncertainty model and measure model.

A.1 Belief Model

The Belief Model in Figure 14 takes the subjective way to represent uncertainty – Uncertainty is a subjective phenomenon that is indelibly bound to the worldview held by a belief agent, – that, for whatever reason, is incapable of possessing complete and fully accurate knowledge about some subject of interest [15]. In addition, the definitions of the concepts in Belief Model are represented in Table 7.

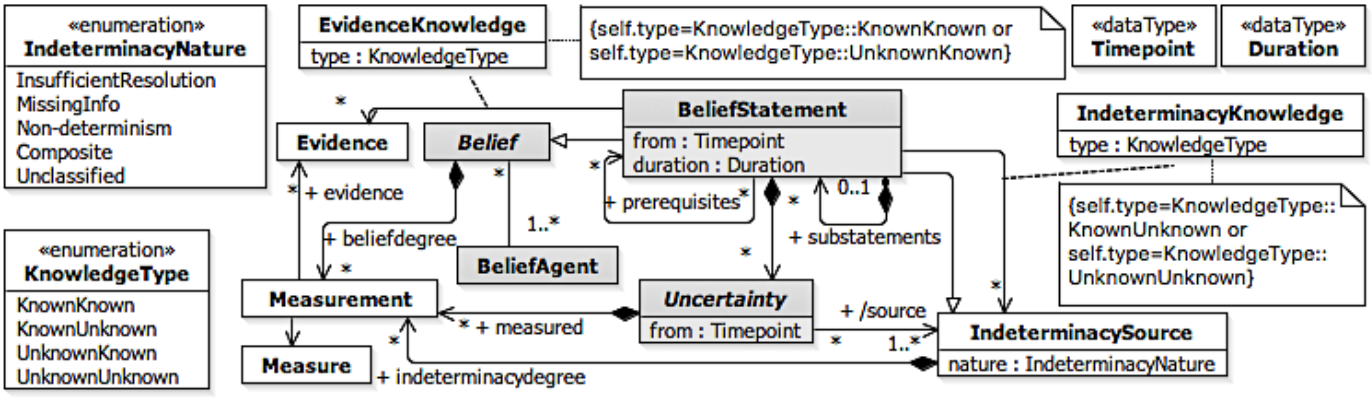


Figure 14. Core Belief Model of *U-Model*

Table 7. Definitions of the Concepts in the Belief Model

Concept	Definition
Belief (abstract)	A belief is an *implicit* subjective explanation or description of some phenomena or notions[8] that is held by a *BeliefAgent*. **Semantics**: This is an *abstract concept* whose only concrete manifestation is in the form of a belief statement. **Features**: • beliefdegree [*] – The *Measurement* of *Belief* derived from *Measurement* of *Uncertainties* in this *Belief*. • beliefAgents [1..*] – The set of *BeliefAgent* held this *Belief*.
BeliefAgent	*BeliefAgent* represents an individual, a community of individuals sharing the same set of beliefs, or a technology, such as a software system, with built-in beliefs. **Semantics**: A belief agent is a physical entity[9] that holds (i.e., owns) one or more beliefs about phenomena or notions associated with one or more subject areas derived from Indeterminacy. This could be a human individual or group, an institution, a living organism, or even a machine such as a computer. Crucially, a belief agent is capable of actions based on its beliefs. **Features**:

[8] The term "phenomena" here is intended to cover aspects of objective reality, whereas "notion" covers abstract concepts, such those encountered in mathematics or philosophy.

[9] We exclude here from this definition "virtual" belief agents, such as those that might occur in virtual reality systems and computer games.

Concept	Definition
	• beliefs [*] – The set of *Belief* that represent the full set of beliefs held explicitly by the *BeliefAgent*.
BeliefStatement	A *BeliefStatement* is an *explicit* specification of some *Belief* about a possible phenomenon or notions belonging to a given subject area. **Generalizations**: *Belief*, *IndeterminacySource* **Semantics**: The concrete form of this statement can vary, and may represent informal pronouncements made by individuals or groups, documented textual specifications expressed in either natural or formal languages, formal or informal diagrams, etc. Since it represents a belief, which is a subjective concept, a *BeliefStatement* may not necessarily correspond to objective reality. This means that it could be completely false, or only partially true, or completely true. However, due to the complex nature of objective reality, it may not always be possible to determine whether or not a *BeliefStatement* is valid. Furthermore, the validity of a statement may only be meaningfully defined within a given context or purpose. Thus, the statement that "the Earth can be represented as a perfect sphere" may be perfectly valid for some purposes but invalid or only partly valid for others. For our needs, we are less interested in the validity of a *BeliefStatement* than we are in the level of *Uncertainty* that a belief agent associates with it. **Features**: • substatements [*] – The set of finer-grained *BeliefStatements* that are components of a composite *BeliefStatement*. • prerequisites [*] – The set of *BeliefStatement* on which this *BeliefStatement* depends. • indeterminacySource [*] – The set of *IndeterminacySource* that this *BeliefStatement* involves. • evidence [*] – The set of *Evidence* providing this *BeliefStatement*. • uncertainties [*] – The set of expressions of uncertainty that qualify and/or quantify the degree to which the *BeliefAgent* lacks confidence in this BeliefStatement; this attribute provides the core link between the Belief portion and the Uncertainty portion of the core uncertainty model. • from [0..1] – The *Timepoint* when *BeliefStatement* is initialized. • duration [0..1] – The *Duration* when *BeliefStatement* is active.
Evidence	*Evidence* is either the observation of or record of a real-world event occurrence or, alternatively, the conclusion of some formalized chain of logical inference, which provides information that may contribute to determining the validity (i.e., truthfulness) of a *BeliefStatement*. **Semantics**: *Evidence* is fundamentally an objective phenomenon, representing *something that actually happened*. This means that we do exclude here the possibility of counterfeit or invented evidence. Nevertheless, although *Evidence* represents objective reality, it need not be conclusive in the sense that it removes all doubt (uncertainty) about a *BeliefStatement*. On the other hand, any valid *BeliefStatement* must have at least some *Evidence* to support it.
EvidenceKnowledge	*EvidenceKnowledge* expresses an objective relationship between a *BeliefStatement* and relevant *Evidence*. **Semantics**: *EvidenceKnowledge* identifies whether the corresponding *BeliefAgent* is aware of the appropriate *Evidence*. Thus, an agent may be either aware that it knows something (*KnownKnown*), or it may be completely unaware of *Evidence* (*UnknownKnown*).
IndeterminacyNature	*IndeterminacyNature* represents the kind of indeterminacy[10]. **Enumeration literals:** • InsufficientResolution – The information available about the phenomenon in question is not sufficiently precise. • MissingInfo – The full set of data is unavailable at the time the statement is made. • Non-determinism – The phenomenon in question is either practically or inherently non-deterministic. • Composite – This represents some combination of multiple other kinds of indeterminacy. • Unclassified – Indeterminate indeterminacy.
IndeterminacySource	*IndeterminacySource* represents a situation whereby the information required to ascertain the validity of a *BeliefStatement* is indeterminate in some way, resulting in uncertainty

[10] Indeterminacy represents a situation whereby the full knowledge necessary to determine the required factual state of some phenomena or notions is unavailable.

Concept	Definition
	being associated with that statement. **Semantics**: One possible source of indeterminacy could be another *BeliefStatement*. A given indeterminacy source could in some cases be decomposed into more basic sources. **Features**: • indeterminacydegree [*] – This set of Measurement represents the quantification (or qualification) of this IndeterminacySource. • nature [1] – The *IndeterminacyNature* represents the kind of indeterminacy reason.
IndeterminacyKnowledge	*IndeterminacyKnowledge* expresses an objective relationship between an *IndeterminacySource* and the awareness that the *BeliefAgent* has of that source. **Semantics**: *IndeterminacyKnowledge* identifies whether the corresponding *BeliefAgent* is aware of the appropriate *IndeterminacySource*. So, even though it is agent specific, it is still an objective concept since it does not represent something that is declared by the agent. For instance, an agent may be aware that it does not know something about a possible source (*KnownUnknown*), or the agent may be completely unaware of a possible source of indeterminacy (*UnknownUnknown*).
KnowledgeType	*KnowledgeType* represents the type of the knowledge. **Enumeration literals:** • KnownKnown – Indicates that an associated *BeliefAgent* is consciously aware of some relevant aspect. • KnownUnknown (Conscious Ignorance) – Indicates that an associated *BeliefAgent* understands that it is ignorant of some aspect. • UnknownKnown (Tacit Knowledge) – Indicates that an associated *BeliefAgent* is not explicitly aware of some relevant aspect that it, nevertheless, may be able to exploit in some way • UnknownUnknown (Meta Ignorance) – Indicates that an associated *BeliefAgent* is unaware of some relevant aspect.
Measure	*Measure* represents the way of measuring *Belief/Uncertainty/IndeterminacySource*. **Semantics**: *Measure* is *objective concept*, and specifies the existing way/theory to measure uncertainty.
Measurement	*Measurement* represents the optional quantification (or qualification) that specifies the degree of *Belief/Uncertainty/IndeterminacySource*. **Semantics**: It may be possible to specify a *Measurement* that quantifies in some way (e.g., as a probability or a percentage) the degree of *Uncertainty* by the agent making the belief statement. Note, however, that this is a subjective measure defined by the *BeliefAgent*. **Features**: • measure [1] – This *Measure* represents the related way of measuring *Belief/Uncertainty/IndeterminacySource*.
Uncertainty	*Uncertainty* (lack of confidence) represents *a state of affairs whereby a BeliefAgent does not have full confidence in a Belief that it holds*. **Semantics**: "Full confidence" here means that the agent does not have any doubts about the validity of a statement. It is important to distinguish here between certainty and validity. That is, an agent could have full confidence in a *BeliefStatement* that is actually false; i.e., a statement that does not match (objective) truth. In general, the source of uncertainty associated with a *BeliefStatement* is that, for some reason, the agent does not have full knowledge of all relevant facts pertaining to the phenomena or notions that are the subject of the statement. **Features**: • from [0..1]– The *Timepoint* when *Uncertainty* is initialized. • measured [*]– This *Measurement* is used for representing confidence degree of *Uncertainty* by the agent making the *BeliefStatement*. • source [1..*]– This set of *IndeterminacySource* derived from the involves association and generalization of *BeliefStatement*.

A.2 Uncertainty Model

The Uncertainty Model in Figure 15 inspired by the literature of uncertainty expands on Uncertainty from several different viewpoints and introduces related abstractions [15], i.e. risk, pattern, and the definitions of the concepts in Uncertainty Model are represented in Table 8.

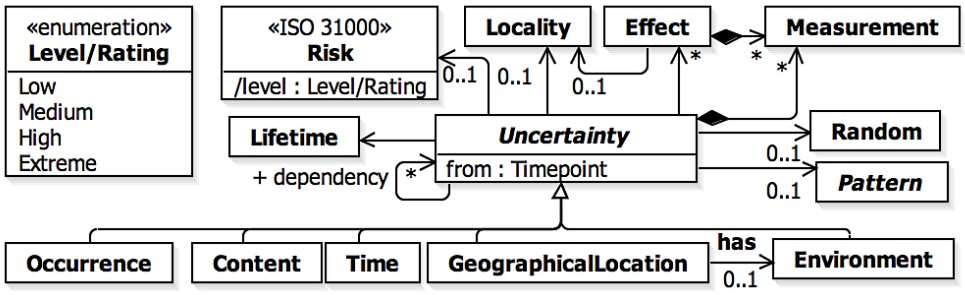


Figure 15. Core Uncertainty Model of *U-Model*

Table 8. Definitions of the Concepts in the Core Uncertainty Model

Concept	Definition
Effect	*Effect* represents the result of *Uncertainty* in the *BeliefStatement*. **Semantics**: An uncertainty may result into: 1) another known *Uncertainty*, 2) something known and is not *Uncertainty*, 3) anything unknown. **Features**: • locality [0..1] – This value is used to represent that the *Locality* of the *Effect*. • measurement [*] – This set of *Measurement* represents the quantification (or qualification) of this *Effect*.
Lifetime	*Lifetime* represents the duration of time for which an *Uncertainty* remains active. **Semantics:** The length of time for which *Uncertainty* exists. For example, an *Uncertainty* may appear temporarily for a short period of time and disappears itself. On the other hand, an *Uncertainty* could be persistent, i.e., it stays active until appropriate actions are taken to resolve the *Uncertainty*.
Locality	A particular place or a position where *Uncertainty* occurs in the *BeliefStatement*. **Semantics:** A location could be a geographical location or a position where *Uncertainty* occurs. The concept of location is different than the *Uncertainty* type *GeographicalLocation*, where *Uncertainty* is due to the geographical location, however in this concept *Uncertainty* occurred at a location may not be due to the geographical location.
Pattern	*Pattern* represents an intelligible way in which an *Uncertainty* appears. **Semantics:** An *Uncertainty* may occur without any *Pattern*, i.e. *Random*, or may have a pattern in which it may occur, for example, occurring at equal intervals of time, i.e., Periodic.
Random	An *Uncertainty* that occurs without definite method, purpose or conscious decision. **Semantics**: An *Uncertainty* occurring without any specific pattern.
Risk	*Risk* measures the risk associated with *Uncertainty*. **Semantics:** An uncertainty may have an associated risk and high-risk uncertainties deserve special attention.
Level/Rating	*Level/Rating* is derived from *Measurement* owned by *Uncertainty* (Probability of the Occurrence of an Uncertainty) and Measurement owned by Effect (e.g., high impact), for example, using the risk matrix [40] or any other matrices
Occurrence	*Occurrence* represents a situation whereby a *BeliefAgent* lacks confidence in occurrence existing in a *BeliefStatement*. **Generalizations**: *Uncertainty*
Content	*Content* represents a situation whereby a *BeliefAgent* lacks confidence in content existing in a *BeliefStatement*. **Generalizations**: *Uncertainty*
Time	*Time* represents a situation whereby a *BeliefAgent* lacks confidence in time existing in a *BeliefStatement*. **Generalizations**: *Uncertainty*
GeographicalLocation	*GeographicalLocation* represents a situation whereby a BeliefAgent lacks confidence in

Concept	Definition
	geographical location existing in a *BeliefStatement*. **Generalizations**: *Uncertainty*
Environment	*Environment* represents a situation whereby a *BeliefAgent* lacks confidence in environment existing in a *BeliefStatement*. **Generalizations**: *Uncertainty*
Uncertainty	**Semantics**: The uncertainty model expands on *Uncertainty* from several different viewpoints and introduces related abstractions. Notice that Uncertainty has a self-association. This self-association facilitates: 1) relating different *Application level* uncertainties to each other, 2) relating different *Infrastructure level* uncertainties to each other, 3) relating *Application level* and *Infrastructure level* uncertainties to each other, 4) relating *Integration level* uncertainties to each other, and 5) relating *Application*, *Integration*, and *Infrastructure level* uncertainties. This self-association can be specialized into different types of relationships such as ordering and dependencies. Here, we intentionally did not specialize it to keep the model general, so that it can be specialized for various purposes and contexts. **Features:** • lifetime [1]– This value is used for representing the duration of this *Uncertainty*. • pattern [0..1]– This value is used for describing whether this *Uncertainty* happens in a pattern or what kind of the pattern this *Uncertainty* occurs in. • risk [0..1]– This value is used for whether this *Uncertainty* has a risk, and what kind of risk this *Uncertainty* causes. • locality [0..1] – This value is used for representing what location this *Uncertainty* occurs. • effect [*]– This value is used for describing what effect the uncertainty may produce. • dependency [*]– The set of *Uncertainty* represents the dependency relationship with other *Uncertainty*.

The Pattern Model in Figure 16 shows the conceptual model for the occurrence pattern of Uncertainty [15], and the definitions of the concepts in Pattern Model are represented in Table 9.

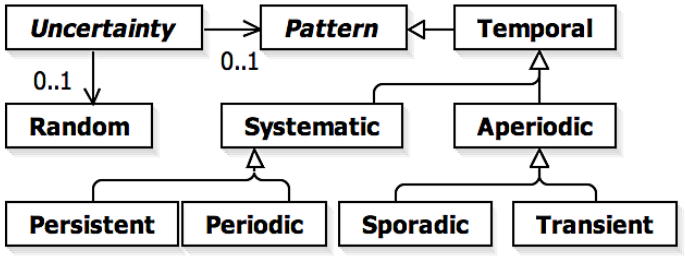


Figure 16. The Pattern Model

Table 9. Definitions of the Concepts in the Pattern Model

Concept	Definition
Temporal	*Uncertainty* occurring in a temporal pattern. **Generalizations**: *Measure* **Semantics**: *Temporal* describes the notion of time with the occurrence of uncertainty.
Systematic	*Uncertainty* occurring in a systematic pattern. **Generalizations**: *Temporal* **Semantics**: *Uncertainty* occurring in some methodical pattern, i.e., a pattern that can be described in a mathematical way.
Persistent	A permanent *Uncertainty*, i.e., lasting forever.

Concept	Definition
	Generalizations: *Systematic* **Semantics** The definition of "forever" varies. For example, an uncertainty may exist permanently until appropriate actions are taken to deal with the uncertainty. On the other hand, an uncertainty may not be able to resolve and stays forever.
Periodic	An *Uncertainty* that occurs in repeated periods or at regular intervals. **Generalizations**: Systematic **Semantics**: *Uncertainty* repeating itself after an equal interval of time.
Aperiodic	An *Uncertainty* that occurs at irregular intervals of time. **Generalizations**: *Temporal* **Semantics**: It is important to note that *Aperiodic* is inherited from *Temporal*; this means it has a notion of time in which the *Uncertainty* appears in an *Aperiodic* pattern.
Sporadic	*Uncertainty* occurring in a sporadic pattern. **Generalizations**: *Aperiodic* **Semantics**: *Uncertainty* occurring occasionally.
Transient	*Uncertainty* occurring temporarily. **Generalizations**: *Aperiodic* **Semantics**: *Uncertainty* that does not last long.

A.3 Measure Model

The Measure Model in Figure 17 describes the commonly known measures related to Uncertainty [15], and the definitions of the concepts in Measure Model are represented in Table 10.

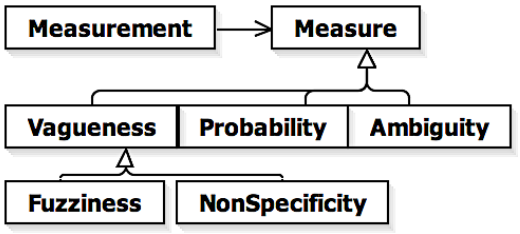


Figure 17. Core Measure Model of *U-Model*

Table 10. Definitions of the Concepts in the Core Measure Model

Concept	Definition
Vagueness	*Uncertainty* measured with the vagueness methods. **Generalizations**: *Measure*
Fuzziness	*Uncertainty* measured by fuzzy methods. More details can be referred to the fuzzy logic literature [41]. **Generalizations**: *Vagueness*
NonSpecificity	*Uncertainty* measured using non-specificity methods. **Generalizations**: *Vagueness* **Semantics**: In certain cases, it may not be possible to measure an uncertainty using quantitative measurements and instead qualitative measurements can be used. Such qualitative measurements are classified under *Non-Specificity* methods.
Probability	*Uncertainty* measured with the probability. **Generalizations**: *Measure* **Semantics**: A quantitative way of measuring uncertainty.
Ambiguity	*Uncertainty* in the *BeliefStatement* is measured using ambiguity way. **Generalizations**: *Measure* **Semantics**: An uncertainty may be described ambiguously. For example, in the following statement": The food is hot", the ambiguity is in the measurement, i.e., the food is either hot

	in terms of temperature or in terms of spices.

Appendix B OCL Constraints for Restriction Rules

In Table 11, we provided restriction rules specified with OCL based on *BeliefUCMeta*.

Table 11. New and Extended Restriction Rules Specified with OCL

#	OCL Constraints
UR16	**context** BeliefSpecialSentence **inv:** (**self.oclIsTypeOf**(UCMeta::UCSTemplate::SpecialSentence::IncludeSentence) **and self**.uncertainty->size() = 1) **implies** (**self**.kind = UncertaintyKind::Occurrence)
UR17	**context** BeliefSpecialSentence **inv:** (**self.oclIsTypeOf**(UCMeta::UCSTemplate::SpecialSentence::ExtendSentence) **and self**.uncertainty->size() = 1) **implies** (**self**.kind = UncertaintyKind::Occurrence)
UR19	**context** BeliefComplexSentence **inv: self.oclIsTypeOf**(UCMeta::UCSTemplate::ComplexSentence::ConditionalSentence) **implies** ((**self**.IFcondition->uncertainty->size()=1 **implies self**.IFcondition->uncertainty->forAll(u:Uncertainty|u.kind = UncertaintyKind::Content)) **and** ((**self**.ELEFcondition->size() = 1 **and self**.ELEFcondition->uncertainty->size()=1) **implies self**.ELEFcondition->uncertainty->forAll(u:Uncertainty|u.kind = UncertaintyKind::Content))
UR20	**context** BeliefComplexSentence **inv:** (**self.oclIsTypeOf**(UCMeta::UCSTemplate::ComplexSentence::ParallelSentence) **and self**.uncertainty->size() = 1) **implies** (**self**.kind = UncertaintyKind::Time)
UR21	**context** BeliefComplexSentence **inv:** (**self.oclIsTypeOf**(UCMeta::UCSTemplate::ComplexSentence::ConditionCheckSentence) and **self**.uncertainty->size() = 1) **implies** (**self**.kind = UncertaintyKind::Occurrence)
UR22	**context** BeliefComplexSentence **inv: self.oclIsTypeOf**(UCMeta::UCSTemplate::ComplexSentence::IterativeSentence) **implies** (**self**.WHILEcondition->uncertainty->size()=1 **implies self**.WHILEcondition -> uncertainty->forAll(u:Uncertainty|u.kind = UncertaintyKind::Content))
UR26	**context** BeliefAgent **inv: self**.name <> **null and** BeliefAgent.allInstance()->forAll(a1,a2:BeliefAgent| a1.name <> a2.name)) **context** Evidence **inv: self**.name <> **null and** Evidence.allInstance()->forAll(a1,a2:Evidence| a1.name <> a2.name)) **context** IndeterminacySource **inv: self**.nature <> **null and** IndeterminacySource.allInstance()->forAll(a1,a2: IndeterminacySource | a1.name <> a2.name))
UR27	**context** IndeterminacySource **inv: self**.nature <> **null**
UR28	**context** BeliefUseCaseSpecification **inv: self**.agents->size()>0
UR29	**context** BeliefUseCaseSpecification **inv: self**.from <> **null and self**.duration <> **null**
UR30	-- The format check was implemented with java using regular expression.
UR31	-- Manually implemented by users.
UR32	-- The format check was implemented with java using regular expression.
UR33	**context** BeliefClassifier **inv:** (**self**.uncertainty->size()> 0) **implies (self**.beliefDegree <> **null and self**.beliefDegree.value < 1.0) **and** (**self**.measurement->size()>0 **implies self**.measurement-> forAll(m1,m2:Measurement| m1.kind <> m2.kind)) **context** Uncertainty **self**.measurement->size()>0 **implies self**.measurement-> forAll(m1,m2:Measurement| m1.kind <> m2.kind))
UR34	-- The format check was implemented with java using regular expression.
UR35	**context** BeliefSentence **inv: self**.uncertainty->size()>0 **implies self**.uncertainty-> forAll(u:Uncertainty| u.**oclAsType**(NLUncertainty)<> **null and self**.content.contains(u.**oclAsType**(NLUncertainty).nl)) **context** Uncertainty **inv: self**.kind <> **null**
UR36	**context** MeasurementStatement **inv: self**.uncertainty->forAll(u:Uncertainty|u.measurement->forAll(m:Measurement|m.uncertainty->size() = 0))
UR37	**context** BeliefUseCaseSpecification **inv: self**.flows->forAll(f:UCMeta::UCTemplate::FlowOfEvents|**self**.ownedKnowledge->includesAll(f.**oclAsType**(BeliefFlowOfEvents).knowledge) **and** f.steps->forAll (bs:UCMeta::UCTemplate::Sentence|**self**.ownedKnowledge->includesAll(bs.**oclAsType**(BeliefSentence).knowledge) **and** (bs.**oclAsType**(BeliefSentence). uncertainty->size() > 0 **implies** bs.**oclAsType**(BeliefSentence). uncertainty->forAll(u:Uncertainty|**self**.ownedKnowledge->includesAll(u.knowledge)))))
UR38,39	-- The format check was implemented with java using regular expression.
UR40	**context** BeliefAlternativeFlow **inv: (self**.rfs->size()=0 **and self**.urfs->size()>0 **implies** (**self**.urfs-> forAll (s:UCMeta::UCTemplate::Sentence|s.**oclAsType**(BeliefSentence).uncertainty->size()>0) **and self**.altSteps->size()>0 **and self**. altSteps-> select(a:AlternativeSentence| a.urfs->size()>0)->size()>0 **and self**.altSteps->forAll(a:AlternativeSentence|self.urfs-> includesAll(a.urfs)))) **and** (**self**.rfs->size()>0 **and self**.urfs->size()=0 **implies self**.rfs->

	`forAll(s:UCMeta::UCTemplate::Sentence|s.oclAsType(BeliefSentence).uncertainty->size()=0))`
UR41	`context BeliefAlternativeFlow` `inv: self.rfs->size()=0 and self.urfs->size()>0 self. altSteps-> select(a:AlternativeSentence| a.urfs->size()>0)->size()>0 and self.altSteps->forAll(a:AlternativeSentence|self.urfs-> includesAll(a.urfs))`
UR42	`context NLUncertainty` `inv: self.nl<>'IF' and self.nl<>'THEN' and self.nl<>'ELSE' and self.nl<>'ELSEIF' and self.nl<>'ENDIF' and self.nl<>'DO' and self.nl<>'UNTIL' and self.nl<>'ABORT' and self.nl<>'RESUME STEP' and self.nl<>'RFS' and self.nl<>'URFS'`

Appendix C An Example of Questionnaire of the AW Case Study

As discussed in Section 6.2, we conducted a questionnaire-based survey. The summary of the questions is provided in Table 13 where we also indicate the example questions that are relevant to each type of the question. In Section C.1, we provide a sanitized uncertainty requirement of the AW case study, for which a list of questions was derived (Q1-Q10). In Section C.2, after refining the RUCM specification (Figure 18), two questions (Q11-Q12) were derived.

Table 12. Design of the Questionnaire and Example Questions

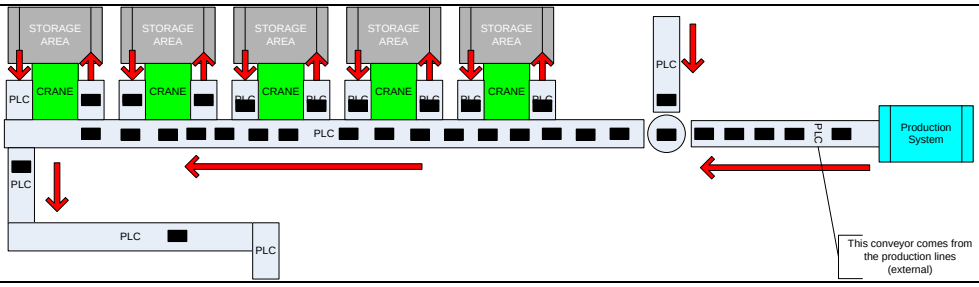
#	Explanation	Index of Question
1	Inquiry the boundary of the system to define actors in the use case model.	Q1, Q2, Q3
2	Check the completeness of the flow of events of each use case specification.	Q4
3	Inquiry the existence of sources related to an actor.	Q5, Q6, Q7
4	Inquiry the existence of potential uncertainties related to system properties or behaviors.	Q7
5	Inquiry existence of the potential uncertainties regarding time, nature and human being.	Q8, Q9, Q10
6	Inquiry if a potential uncertainty is valid by checking if it is derived based on system properties or behaviors.	Q11-Q12.1)
7	Inquiry the completeness of the types of uncertainties defined in U-Model.	Q11-Q12.2)
8	Inquiry the type of a specified uncertainty.	Q11-Q12.2)
9	Inquiry the measure and measurement of a specified uncertainty.	Q11-Q12.3).a)
10	Inquiry the risk of a specified uncertainty.	Q11-Q12.3).b)
11	Inquiry the evidence to support the specified measurement and risk of a specified uncertainty.	Q11-Q12.3).a-b).i

C.1 Examples of Uncertainty Requirements Specified in RUCM and Corresponding Questions

In Table 13, we provide an example of uncertainty requirements developed by our industrial partner. Note that the RUCM editor was not used because the partner wanted to add additional fields (e.g., *Means for validation/verification*, *Models/Metrics*, *Change History*), which were not part of the RUCM template. Therefore, Word was used as the tool to specify all the use case specifications.

Table 13 An Example of uncertainty requirements specified in RUCM

Scale Up for a Larger Number of Orders to Handle	
Use Case ID	UC2_INTE_1.1
Use Case Name	Scale Up for a Larger Number of Orders to Handle
Means for validation /verification	A communication infrastructure among WMS, Production System Simulator and PLC Simulator is built: • The input buffer status is full or not modifying ad-hoc its status. • The Production System Simulator introduces a pallet.

Models/Metrics		
Precondition	The warehouse processes a limited (bounds set by design) number of pallets.	
Primary Actor	-	
Secondary Actors	-	
Dependencies	-	
Generalization	-	
Basic Flow	**Steps**	
	1	The Production System introduces a pallet into the warehouse by the production system.
	2	WMS **VALIDATES THAT** the input buffer has free space to store pallets.
	3	WMS sends the order to enter the new pallet into the input buffer.
	Post-condition: The pallet is located at the input buffer.	
Specific Uncertainty Alternative Flow (buffer hasn't got free space)	**RFS BF 2**	
	1	WMS **VALIDATES THAT** the input buffer has not got any free space to store pallets.
	2	The Pallet waits indefinitely for free space in the buffer.
	3	**ABORT**
	Post-condition: Material flow stops are propagated upstream towards the production system.	
Additional Information	-	
Responsibility	Person A	
Change History	2015-05-20 Person B – First Version 2015-06-01 Person A – Reviewed 2015-07-28 Person A – Refined Version	

Q1. What do you believe about **Production System** mentioned in Table 13?
Actor (Third party) ☐ or Part of Handling system ☐

Q2. What do you believe about **Pallet** mentioned in Table 13?
Actor (Third party) ☐ or Part of Handling system ☐

Q3. What do you believe about **WMS** mentioned in Table 13?
Actor (Third party) ☐ or Part of Handling system ☐

Q4. Do you believe the reason/source of **Specific Uncertainty Alternative Flow** is unknown?
Yes, I have no idea at all ☐
Yes, I have some uncertain idea, but not complete. ☐ Please specify what you know.

No, I know it. ☐ Please specify what you know.

Q5. Do you believe the size of **pallet** can introduce ***uncertainty***?
Yes ☐ No ☐
If no, does any other property of **pallet** can cause uncertainty? Please specify if possible.

Q6. Do you believe that **Production System** introduces the **pallet** always one by one?
Yes ☐ No ☐

Q7. Please refer to step 3 in Table 13, "WMS sends the order to enter the new pallet into the input buffer".

1) Is it possible that "WMS does not send the order to enter the new pallet into the input buffer at all"?
   If yes, the reason is related to
   WMS ☐ Pallet ☐ Input buffer ☐ none of them, please specify
2) Is it possible that "WMS sends the order to enter the new pallet into the input improperly"?
   If yes, the reason is related to
   WMS ☐ Pallet ☐ Input buffer ☐ none of them, please specify
3) Is there any other possibility?
   If yes, please specify

Q8. Do you believe any ***Time constraints*** may cause uncertainty in this case?
If yes, please specify
If yes, do you believe it is necessary to consider in this use case? Yes ☐ No ☐

Q9. Do you believe any ***Nature phenomena*** may cause ***uncertainty*** in this case?
If yes, please specify
If yes, do you believe it is necessary to consider in this use case? Yes ☐ No ☐

Q10. Do you believe any ***human behavior*** may cause ***uncertainty*** in this use case?
If yes, please specify
If yes, do you believe it is necessary to consider in this use case? Yes ☐ No ☐

C.2 Example of Refined Uncertainty Requirements in RUCM and Corresponding Questions

Belief Specification

Belief Spec. Name	Scale up for a large number of Orders to Handle
Brief Description	None
Precondition	None
Primary Actor	Production System
Secondary Actors	Pallet, Input buffer

Basic Flow (Untitled) ▼	Steps	
	1	The Production System asks to introduce a pallet into warehouse to WMS.
	2	WMS VALIDATES THAT the input buffer has free space to store the pallets.
	3	WMS sends the response to Production System.
	4	The Production System introduce a pallect into warehouse.
	5	WMS sends the order to enter the new pallet into the input buffer.
	Postcondition	The pallet is located at the input buffer.

Figure 18 Uncertainty Requirement specified in RUCM (a revised version of Table 13)

Please refer to Figure 18,

Q11. Do you believe that uncertainties exist in **Step 1**?

1) Yes ☐ No ☐
   If yes, how many ***uncertainties*** do exist?
   Please specify these ***uncertainties***, ,
2) For each uncertainty, please specify its type?
   Occurrence ☐ *Content* ☐ *Time* ☐ *Environment* ☐ *Geographical Location* ☐ *Others* ☐
3) Do you believe this ***uncertainty*** can be measured?
   Yes ☐ No ☐ If no, please specify the reason.
   If yes, please answer the following question.

a. What do you believe the likelihood of this ***uncertainty*** is?
Please specify the probability if possible, otherwise please select one of the following options:

Rare	Unlikely	Possible	Likely	Almost Certain
☐	☐	☐	☐	☐

i. Do you have any evidence to support?
Yes ☐ No ☐ if yes, please specify .

b. What level of risk is associated with this type of ***uncertainty***?
Please specify the exact percentage (0-100, 100 is the highest level of risk) if possible; otherwise please select one of the options:

Very Low	Low	Medium	High	Extreme
☐	☐	☐	☐	☐

i. Do you have any evidence to support?
Yes ☐ No ☐ if yes, please specify .

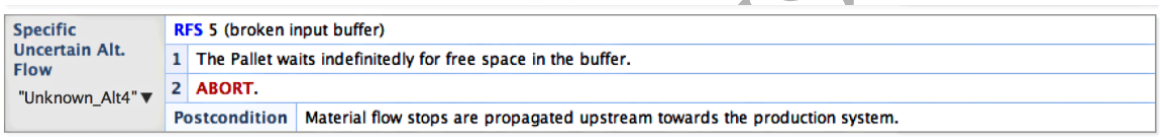


Figure 19 Uncertainty RFS 2

Q12. Please refer to Figure 19, did you specify this ***uncertainty*** as above?

1) Yes ☐ No ☐
If no, do you believe it is real? Yes ☐ No ☐. If no, please specify the reason
If it is real, is it necessary to consider? Yes ☐ No☐. If no, please specify the reason
If it is real and it is necessary to consider, please answer the following
2) For each uncertainty, please specify its type?
Occurrence ☐ *Content* ☐ *Time* ☐ *Environment* ☐ *Geographical Location* ☐ *Others* ☐
3) Do you believe this ***uncertainty*** can be measured?
Yes ☐ No ☐ If no, please specify the reason
If yes, please answer the following question.

a. What do you believe the likelihood of this ***uncertainty*** is?
Please specify the probability if possible, otherwise please select one of the following options:

Rare	Unlikely	Possible	Likely	Almost Certain
☐	☐	☐	☐	☐

i. Do you have any evidence to support?
Yes ☐ No ☐ if yes, please specify .

b. What level of risk is associated with this type of ***uncertainty***?
Please specify the exact percentage (0-100, 100 is the highest level of risk) if possible; otherwise please select one of the options:

Very Low	Low	Medium	High	Extreme
☐	☐	☐	☐	☐

ii. Do you have any evidence to support?

Yes ☐ No ☐ if yes, please specify .

Appendix D A Sanitized Belief Use Case Specification of the AW Case Study Specified with the U-RUCM Editor

Belief Use Case Specification

Use Case Name	Scale Up for a Larger Number of Orders to Handle
Brief Description	None
Primary Actor	Production System
Secondary Actors	Input buffer,Pallet, Operator
Dependency	INCLUDE USE CASE WMS Verifies the Input Pallet
Generalization	None
Belief Agent(s)	CompanyA
Timepoint and Duration	March-2016, After
Belief Degree	UModel.Measure.Vagueness::Likely
Indeterminacy Source(s)	REF Broken Input Buffer, REF Connection Loss, REF Improper Implementation for Adaption of Third part Application
Evidence	REF Execution Log
Belief Precondition	None

Belief Basic Flow "Normal"	Steps	
	1	Production System executes the input order.
	2	DO
	3	Production System inquires of introduction of a pallet into warehouse.
	4	WMS VALIDATES THAT the input buffer has free space to store pallets.
	5	WMS sends the introduction response to Production System.
	6	Production System introduces a pallet.
	7	INCLUDE USE CASE WMS Verifies the Input Pallet
	8	WMS sends the order to MFC for entering the new pallet into the input buffer.
	9	INCLUDE USE CASE MFC Handles Input Order
	10	UNTIL No more introduction inquire.
	11	WMS displays the order is finished.
	Postcondition	The order is finished.

Belief Specific Alternative Flow (URFS) "BAlt1"	URFS Normal 1	
	A1	Production System executes the input order improperly.
	1	WMS watis for the input pallet.
	2	ABORT.
	Postcondition	None

Belief Specific Alternative Flow (RFS) "BAlt2"	RFS Normal 3	
	1	WMS waits for the free space.
	2	WMS VALIDATES THAT the input buffer has free space to store pallets.
	3	RESUME STEP Normal 5
	Postcondition	The input buffer has free space.

Belief Specific Alternative Flow (URFS) "BAlt3"	URFS BAlt2 1	
	A1	WMS displays the time out of waiting for the free space.
	1	Operator checks the input buffer.
	2	ABORT.
	Postcondition	The order is not finished.

Belief Specific Alternative Flow (URFS) "BAlt4"	URFS Normal 6	
	A1	Production System displays the time out of introducing the pallet.
	1	Operator checks the problem manually.
	2	ABORT.
	Postcondition	The order is not finished.

Belief Specific Alternative Flow (URFS) "BAlt5"	URFS Normal 8	
	A1	WMS sends the pallet to the rejection station.
	1	RESUME STEP Normal 2
	Postcondition	The pallet is rejected.

Figure 20. Belief Use Case Specification *Scale up for a large number of orders*

Biography

Man Zhang is a PhD student at Simula Research Laboratory, Norway (2015 - present) and the Department of Informatics, University of Oslo, Norway. Previously, she obtained her Master degree in Computer Technology from Beihang University, Beijing, China (2012 - 2015). Her main research interests include Model-based Testing of Cyber-Physical Systems, Uncertainty Modeling, Model-based Engineering and Requirements Engineering.

Tao Yue is a chief research scientist at Simula Research Laboratory, Oslo, Norway. She has received the Ph.D. degree in the Department of Systems and Computer Engineering at Carleton University, Ottawa, Canada in 2010. Before that, she was an aviation engineer and system engineer for seven years. She has nearly 20 years of experience of conducting industry-oriented research with a focus on Model-Based Engineering (MBE) in various application domains such as Avionics, Maritime and Energy, Communications, Automated Industry, and Healthcare in several countries including Canada, Norway, and China. Her present research area is software engineering, with specific interests in Requirements Engineering, MBE, Model-based Testing, Uncertainty-wise Testing, Uncertainty Modeling, Search-based Software Engineering, Empirical Software Engineering, and Product Line Engineering, with a particular focus on large-scale software systems such as Cyber-Physical Systems. Dr. Yue has been on the program and organization committees of several international conferences (e.g., MODELS, RE, SPLC). She is also on the editorial board of Empirical Software Engineering. Dr. Yue is leading the standardization effort of Uncertainty Modeling at OMG and also actively participating in defining international standards in Object Management Group (OMG) such as SysML and UTP.

Shaukat Ali is currently a senior research scientist in Simula Research Laboratory, Norway. His research focuses on devising novel methods for Verification and Validation (V&V) of large scale highly connected software-based systems that are commonly referred to as Cyber-Physical Systems (CPSs). He has been involved in several basic research, research-based innovation, and innovation projects in the capacity of PI/Co-PI related to Model-based Testing (MBT), Search-Based Software Engineering, and Model-Based System Engineering. He has rich experience of working in several countries including UK, Canada, Norway, and Pakistan. Shaukat has been on the program committees of several international conferences (e.g., MODELS, ICST, GECCO, SSBSE) and also served as a reviewer for several software engineering journals (e.g., TSE, IST, SOSYM, JSS, TEVC). He is also actively participating in defining international standards on software modeling in Object Management Group (OMG), notably a new standard on Uncertainty Modeling.

Bran Selić is President of Malina Software Corp., a Canadian company that provides consulting services to corporate clients and government institutions worldwide. He is also Director of Advanced Technology at Zeligsoft Limited in Canada and a Visiting Scientist at Simula Research Laboratories in Norway. On the academic side, he is an adjunct at Monash University and the University of Sydney (both in Australia) as well as a long-term lecturer at INSA University in Lyon, France. With over 40 years of practical experience in designing and implementing large-scale industrial cyber-physical systems, Bran has pioneered the application of model-based engineering methods in real-time and embedded applications and has led the definition and adoption of several international standards in that domain, including managing the definition and adoption of the widely used Unified Modeling Language (UML) standard. In addition, he has been a founder and co-founder of several successful technology spin-offs.

Roland Norgren holds a bachelor's degree in system science. He has worked mainly as developer and project manager in both small and large scale organisations. Roland is today working as process manager for research and innovation at the GIS-cluster Future Position X in Gävle, Sweden.

Oscar Okariz is with ULMA since 2010. He is Computer Engineer by the Basque University (UPV) since 2003. Oscar Okariz is recognized by his experience on J2EE and .Net development as well as his experience of more than ten years as software developer analyst on several enterprises managing and directing projects. He is currently "Software technology" area responsible in charge of leading internal team work to achieve a new generation software product in Ulma Handling Systems.

Karmele Intxausti is the Head of Big Data Architectures Research Area at IK4-IKERLAN. She obtained her Computer Science Engineering master degree from the University of the Basque Country in 1981, a Master in Business Administration from University of Deusto in 2002 and a Master in Statistical Learning and Data Mining from the Spanish Open University in 2012. From 1982 to 1986 she worked as a lecturer in the Computer Science Department of the University of the Basque Country. From 1987 to 2001 she worked in the Artificial Intelligence department in Ikerlan and from 2001 to 2015 in the Information Technologies Department. Currently, she is the team leader of the Big Data Architectures team. She has participated in numerous R&D contract-based projects for companies in several sectors: development of Manufacturing Execution Systems, control of containers in ports, Maintenance systems for capital goods, etc. She has also participated in several European Projects and, currently, she is involved in U-Test and in the ECSEL project "MANTIS- Cyber Physical System based Proactive Collaborative Maintenance"